



Tecnológico de Monterrey

Inteligencia artificial avanzada para la ciencia de datos

Reporte

Predicción de retrasos en vuelos mediante
regresión lineal y XGBoost

Mauricio Olguín Sánchez
A01711522

Agosto 2026

Índice general

1. Realizando la implementación «a manita»	1
1.1. Sobre la base de datos seleccionada	1
1.2. Proceso de ETL	2
1.2.1. <i>Extraction</i>	2
1.2.2. <i>Transforming</i>	2
1.2.3. <i>Loading</i>	3
1.3. Análisis exploratorio de los datos	4
1.3.1. Conclusiones	5
1.4. Modelo de regresión lineal sin <i>framework</i>	6
1.4.1. Normalización de los datos	6
1.4.2. Implementación del modelo	6
1.4.3. Resultados del modelo	7
2. Modelo con <i>framework</i>: XGBoost	10
2.1. Modificaciones al ETL	10
2.2. Antecedentes	10
2.3. Selección y configuración de XGBoost	10
2.3.1. Configuración utilizada	11
2.3.2. Significado de los hiperparámetros	11
2.3.3. Técnicas de regularización	12
2.4. Resultados de XGBoost	12
2.4.1. Conclusión sobre sesgo, varianza y ajuste	13
2.4.2. Comparación: Manual vs XGBoost	13
2.5. Mejora final del modelo XGBoost	13
2.5.1. Configuración del modelo mejorado	14
2.5.2. Confirmación de la mejora	15
2.5.3. Comparación final en <i>test</i>	15
2.5.4. Conclusión sobre sesgo, varianza y ajuste	17
2.6. Discusión final	17
2.6.1. Por qué los modelos no fueron suficientes	17
2.6.2. Posibles mejoras	18
2.6.3. Conclusiones finales	18
A. Figuras ampliadas	19
B. Códigos	29

Capítulo 1

Realizando la implementación «a manita»

1.1. Sobre la base de datos seleccionada

Después de investigar en páginas como [UC Irvine](#), [Kaggle](#) y [World Bank Open Data](#), encontré en [OpenML](#) un *dataset* que se me hizo interesante. Tras aplicar varios filtros, seleccioné la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente, de la base de datos *Airline On-Time Performance Data*. La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, la fuente original del *Bureau of Transportation Statistics* ofrece una descripción más detallada de los datos:

DepDelay - numérico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numérico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numérico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numérico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numérico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numérico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

UniqueCarrier - categórico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categórico. Código del aeropuerto de origen del vuelo.

Dest - categórico. Código del aeropuerto de destino del vuelo.

Distance - numérico. Distancia total del vuelo, expresada en millas.

1.2. Proceso de ETL

El proceso de ETL evolucionó conforme cambió el alcance del proyecto. La primera versión estaba diseñada únicamente para la regresión manual: excluía *UniqueCarrier*, *Origin* y *Dest*, representaba *Month*, *DayofMonth* y *DayOfWeek* mediante *One-Hot Encoding* y convertía los horarios HHMM directamente a minutos, sin validar antes que las horas y los minutos fueran posibles. También guardaba una representación reducida orientada a ese modelo.

Al incorporar modelos con *framework*, generalicé el ETL para conservar las variables categóricas y realizar la preparación específica dentro de cada modelo. En la versión definitiva, añadí la validación de horarios, la conversión especial de 2400 a medianoche y la codificación cíclica de las variables de calendario. Las métricas finales de los tres modelos se obtuvieron con esta versión, que se describe en las siguientes subsecciones; el ETL inicial sólo explica la evolución del proyecto y algunas de las gráficas exploratorias.

1.2.1. *Extraction*

Como mencioné anteriormente, obtuve los datos del siguiente enlace: <https://www.openml.org/search?type=data&status=active&id=42728>

Si se quisieran obtener datos más actuales, habría que descargarlos manualmente o crear un *script* para

obtenerlos desde el siguiente enlace: <https://transtats.bts.gov/PREZIP/>. En esta página se recopilan datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

Sobre los atributos *Month*, *DayofMonth* y *DayOfWeek*

Las variables *Month*, *DayofMonth* y *DayOfWeek* se almacenan como números, pero describen ciclos. Por ello, no conviene introducirlas directamente como cantidades lineales: diciembre y enero, o domingo y lunes, son periodos consecutivos aunque sus códigos numéricos se encuentren en extremos opuestos. En el procesamiento actual cada variable x se sustituye por un par de componentes:

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right),$$

donde P vale 12 para el mes, 31 para el día del mes y 7 para el día de la semana. Esta codificación conserva la cercanía entre los extremos de cada ciclo y produce seis predictores numéricos. Se utiliza tanto en el modelo manual como en XGBoost. En el caso del día del mes se utiliza un periodo fijo de 31 días, por lo que la representación es una aproximación para los meses más cortos.

Las variables *UniqueCarrier*, *Origin* y *Dest* se excluyen únicamente del modelo manual para conservar una implementación manejable con listas y ciclos. En XGBoost sí se incorporan mediante una codificación categórica ajustada exclusivamente con los datos de entrenamiento.

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de

datos original, estas se presentan en formato numérico HHMM (por ejemplo, las 13:20 se representan como 1320).

Sin embargo, esta representación no es adecuada para utilizarla directamente en los modelos. El formato HHMM introduce saltos numéricos que no corresponden al tiempo real. Por ejemplo, 1059 (10:59) y 1100 (11:00) están separados por un minuto, pero el modelo podría interpretar una diferencia de 41 unidades. Esto hace que los horarios parezcan más alejados de lo que realmente están.

Antes de la conversión se valida que ambos atributos representen una hora posible: deben ser enteros, contener horas entre 0 y 23 y minutos entre 0 y 59. El valor 2400 se admite como una representación especial de medianoche y se transforma en 0. Si cualquiera de los dos horarios de una observación es inválido, la observación completa se elimina. Esta revisión encontró un único registro con *CRSArrTime*=2096; al no existir evidencia para reconstruir su valor correcto, se eliminó en lugar de imputarlo.

Después de la validación, se separaron las horas y los minutos y se aplicó la siguiente ecuación:

$$\text{Minutos}_{\text{desde la medianoche}} = 60 \times \text{Hora} + \text{Minutos}$$

Así, 1059 se convierte en 659 y 1100 en 660. La transformación elimina la discontinuidad aritmética propia del formato HHMM y coloca los horarios en el intervalo de 0 a 1439 minutos. No debe interpretarse como la duración de un vuelo: las horas son locales y la diferencia entre salida y llegada también puede atravesar la medianoche o zonas horarias distintas.

Sobre el formato del archivo de los datos

Al comenzar la exploración, cada ejecución tardaba aproximadamente 45 segundos solamente en cargar los datos. Buscando una forma de reducir este tiempo, encontré el

formato **parquet**. Este formato está optimizado para trabajar con datos grandes, acelerar su lectura y utilizar mejor el almacenamiento [1].

En la primera etapa, la representación reducida orientada al modelo manual pasó de aproximadamente 385 MiB a 56 MiB y su tiempo de carga bajó de 45 segundos a 2 segundos o menos. Más adelante, el Parquet definitivo aumentó a aproximadamente 119.5 MiB porque conserva las variables categóricas y las transformaciones que requieren los demás modelos. Aun así, sigue siendo considerablemente menor que el archivo ARFF original. El tiempo anterior corresponde únicamente a la versión inicial; no realicé una nueva medición formal para el archivo definitivo.

1.2.3. Loading

Después de eliminar la observación inválida quedaron 9,999,999 registros. En este proyecto quiero evaluar vuelos de la misma población histórica y no simular la predicción de un periodo futuro. Además, el *dataset* no contiene el año ni una fecha completa que permita hacer una separación cronológica correcta. Por estas razones, mezclé los datos con la semilla 42 antes de dividirlos. La división conserva 98 % de los datos para *train*, 1 % para *validation* y 1 % para *test*: 9,799,999, 99,999 y 100,001 observaciones, respectivamente. La diferencia de una observación se debe al redondeo de las proporciones enteras. Los tres conjuntos son disjuntos. [7]

En las pruebas parciales, la fracción de muestreo se aplica solamente a *train*. Los conjuntos de *validation* y *test* permanecen completos para que sus métricas sean estables y comparables entre ejecuciones. *Train* se utiliza para ajustar los parámetros; *validation*, para comparar configuraciones; y *test* no interviene en la selección de hiperparámetros y se utiliza después de congelar las configuraciones.

1.3. Análisis exploratorio de los datos

Para tener un primer acercamiento general a los datos realicé un gráfico de pares o *pair-plot*, que se muestra en la figura 1.1. Las gráficas de esta sección se generaron durante la exploración inicial, antes de terminar el ETL definitivo, por lo que muestran las variables de calendario originales en lugar de su codificación cíclica.

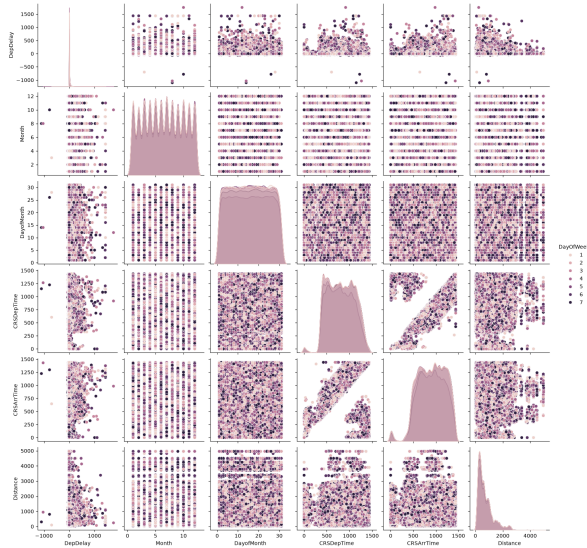
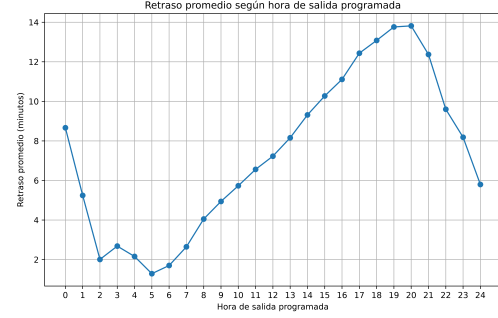


Figura 1.1: *Pairplot* utilizado para comparar las variables. Para reducir el tiempo de cómputo se graficó un millón de observaciones y cada punto se identificó según el día de la semana.

En estas gráficas podemos observar una zona en la que se concentra la mayoría de los valores de *DepDelay*. También aparecen valores atípicos negativos, los cuales representan vuelos que salieron antes de la hora programada. Por otro lado, *CRSDepTime* y *CRSArrTime* presentan agrupaciones claras porque ambas variables representan horarios.

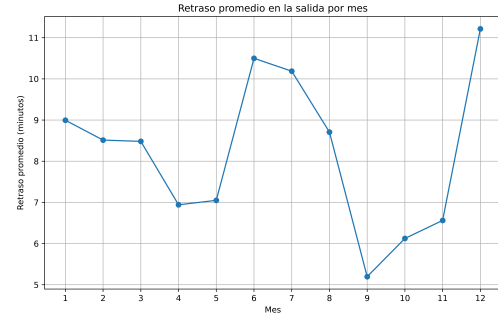
Después de observar el panorama general, realicé gráficas más específicas. La figura 1.2 muestra el retraso promedio según la hora, el día de la semana y el mes.



(a) Promedios de retrasos por cada hora.



(b) Promedios de retrasos por cada día de la semana.



(c) Promedios de retrasos por cada mes.

Figura 1.2: Promedios de retraso según distintos periodos de tiempo.

De estas gráficas, puedo destacar lo siguiente:

- Por hora, el retraso promedio es menor alrededor de las 5 de la mañana y después aumenta hasta alcanzar sus valores más altos entre las 7 y las 8 de la tarde. La categoría 24 pertenece al ETL inicial, que convertía 2400 en 1440 minutos; en la versión definitiva ese valor se interpreta como medianoche y se convierte en 0.
- Por día de la semana, el retraso promedio disminuye del lunes al martes, au-

menta del martes al viernes, alcanza su punto más alto el viernes y desciende el sábado.

- Por mes, los promedios más altos aparecen en junio, julio y diciembre, mientras que el menor se observa en septiembre. Estos periodos pueden coincidir con cambios estacionales o vacaciones, pero con los datos disponibles no puedo afirmar que esa sea la causa.

Estos promedios pueden verse afectados por los valores atípicos. Para observar mejor cómo se distribuyen los retrasos, realicé el histograma de la figura 1.3.

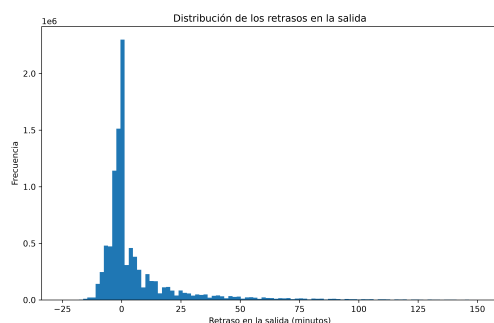


Figura 1.3: Histograma de los retrasos. La escala se recortó de -25 a 150 minutos para observar la concentración central; los valores fuera de este intervalo forman una cola extensa y extrema que reduciría su visibilidad.

La mayoría de los vuelos se concentra cerca de cero minutos de retraso. También existen valores negativos, que corresponden a vuelos que salieron antes de la hora programada.

Por último, utilicé la matriz de correlación de la figura 1.4 para revisar la relación lineal entre las variables. Esta matriz corresponde a la etapa exploratoria y se calculó con las variables de calendario originales, antes de transformarlas en componentes cíclicos.

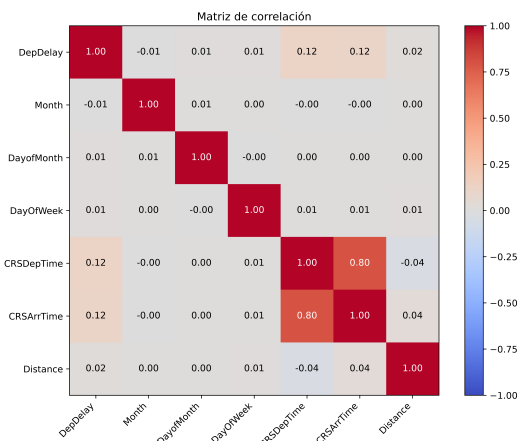


Figura 1.4: Matriz de correlación con las variables originales de la etapa exploratoria.

Como se puede observar, la mayoría de las variables tiene una correlación lineal débil con *DepDelay*. *CRSDepTime* y *CRSArrTime* tienen los valores positivos más altos, de 0.12 en ambos casos, pero la relación sigue siendo pequeña. También existe una correlación de 0.80 entre estas dos variables, algo esperado porque representan los horarios programados de un mismo vuelo.

1.3.1. Conclusiones

A partir de este análisis, puedo concluir lo siguiente:

Naturaleza de los datos: La baja correlación con *DepDelay* indica que una regresión lineal probablemente explicará sólo una pequeña parte de los retrasos.

Multilinealidad temporal: La relación entre las horas de salida y llegada muestra que ambas aportan información parecida, lo que puede afectar la forma en la que el modelo asigna sus pesos.

Diferencia de escalas: Las variables utilizan escalas muy distintas. Por ejemplo, la distancia puede contener miles de millas, mientras que las demás variables tienen valores mucho menores.

Para resolver la diferencia de escalas, normalicé los datos antes de entrenar el modelo

manual. Esto evita que las variables con valores grandes dominen el cálculo del costo y ayuda a que el Descenso de Gradiente encuentre los pesos de una forma más rápida y estable. En la siguiente sección explico cómo realicé esta implementación «a manita».

1.4. Modelo de regresión lineal sin *framework*

1.4.1. Normalización de los datos

Para normalizar los datos utilicé el método *Min-Max*:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Esta transformación coloca las variables utilizadas durante el ajuste dentro de una escala común y facilita la convergencia del descenso de gradiente.

Los valores mínimo y máximo se calculan únicamente con *train*; posteriormente, esos mismos parámetros se aplican a *validation* y *test*. De esta forma se evita que información de los conjuntos de evaluación intervenga en la preparación del modelo. En la implementación manual se normalizan sus nueve predictores numéricos: *CRSDepTime*, *CRSArrTime*, *Distance* y los seis componentes seno y coseno de las variables de calendario. Si una columna de entrenamiento es constante, se sustituye por cero en los tres conjuntos para evitar una división entre cero.

1.4.2. Implementación del modelo

Una vez preparados y normalizados los datos, implementé el modelo de regresión lineal a partir de las variables predictoras. Para una observación con n características, la predicción se expresa como:

$$\hat{y} = b + \sum_{j=1}^n w_j x_j \quad (1.1)$$

donde $\hat{y}$ representa la predicción del modelo, b el bias, w_j el peso asociado a la característica x_j , y n el número total de variables predictoras. A continuación se describen las funciones que componen esta implementación.

Función de hipótesis y cálculo del error

La función de hipótesis recibe las características de una observación junto con los pesos y el bias actuales, y devuelve una predicción:

$$h(x) = b + \sum_{j=1}^n w_j x_j \quad (1.2)$$

Sobre esta predicción se calcula el error de cada observación como la diferencia entre el valor predicho y el real:

$$e_i = \hat{y}_i - y_i \quad (1.3)$$

Tanto la hipótesis como el error son la base del resto de la implementación, ya que el cálculo del costo y de los gradientes dependen directamente de ellos.

Función de costo

Para medir qué tan alejadas se encuentran las predicciones de los valores reales utilicé el *Mean Squared Error* (MSE):

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (1.4)$$

donde m representa el número de observaciones utilizadas. Elevar los errores al cuadrado evita que los errores positivos y negativos se cancelen entre sí, y penaliza en mayor medida los errores de mayor magnitud. El objetivo del Descenso de Gradiente es encontrar los valores de w y b que minimicen esta función.

Gradientes de los parámetros

Para actualizar los parámetros del modelo es necesario conocer cómo cambia la función de costo respecto a cada uno de ellos. Las derivadas parciales del costo respecto a un peso w_j y respecto al bias son:

$$\frac{\partial J}{\partial w_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij} \quad (1.5)$$

$$\frac{\partial J}{\partial b} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (1.6)$$

Ambas expresiones se calculan recorriendo todas las observaciones del lote y acumulando el error correspondiente; la única diferencia es que, en el caso del bias, el error no se multiplica por ninguna característica, ya que la derivada de la hipótesis respecto a b es igual a 1. El resultado de cada gradiente indica la dirección y magnitud en la que debe modificarse el parámetro correspondiente para disminuir el costo.

Actualización de los parámetros

Con los gradientes calculados, los parámetros se actualizan mediante el algoritmo de Descenso de Gradiente:

$$w_j^{\text{nuevo}} = w_j - \alpha \frac{\partial J}{\partial w_j} \quad (1.7)$$

$$b^{\text{nuevo}} = b - \alpha \frac{\partial J}{\partial b} \quad (1.8)$$

donde α representa la tasa de aprendizaje (*learning rate*). El signo negativo es fundamental, ya que el gradiente indica la dirección de mayor incremento del costo; por lo tanto, se avanza en la dirección contraria para acercarse a un mínimo de la función.

Entrenamiento por mini-batch

Debido a que el conjunto completo de entrenamiento contiene aproximadamente 9.8 millones de observaciones, calcular los gradientes utilizando todos los datos antes de

cada actualización resultaría costoso. Por ello utilicé *Mini-Batch Gradient Descent* y dividí las observaciones en lotes de 2048. Esto me permitió actualizar los parámetros varias veces por época y mantener explícitos los ciclos y las listas de la implementación manual.

Durante cada época recorro todos los lotes y repito los siguientes pasos:

1. Realizo una predicción.
2. Calculo el error.
3. Calculo el costo.
4. Calculo los gradientes.
5. Actualizo w y b .
6. Al terminar la época, calculo el costo promedio de todos los lotes.

Uso de los conjuntos de entrenamiento, validación y prueba

El conjunto de *train* es el único utilizado para actualizar los pesos y el bias en cada época. El conjunto de *validation*, en cambio, no participa en esta actualización: después de cada época se usan los parámetros obtenidos hasta ese momento para generar predicciones sobre él y calcular su costo, lo que permite observar qué tanto generaliza el modelo a datos no usados para ajustarlo. El conjunto de *test* se mantiene completamente al margen durante el entrenamiento y solo se utiliza al final, con los parámetros ya definidos, para obtener una evaluación final e independiente del desempeño del modelo.

1.4.3. Resultados del modelo

Para la ejecución final del modelo manual utilicé todos los datos de *train*. Los parámetros que utilicé fueron los siguientes:

- *learning rate* = 0,01
- *epochs* = 20

- *batch size* = 2048
- *random state* = 42

La semilla 42 permite que los pesos iniciales y el orden de los datos sean iguales cada vez que se ejecuta el programa. La curva de la figura 1.5 muestra cómo cambia el error mientras el modelo aprende. Sin embargo, el valor de *train* que aparece en la curva se calcula mientras se actualizan los pesos, por lo que no es exactamente igual al MSE final. Las métricas de la tabla sí se calcularon al final, cuando el entrenamiento ya había terminado.

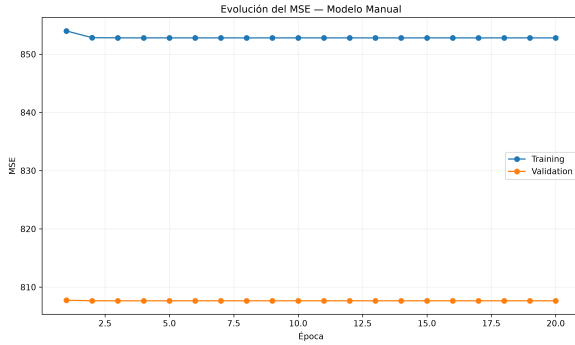


Figura 1.5: Evolución del MSE de entrenamiento y validación del modelo manual durante las 20 épocas.

Las métricas obtenidas con la partición aleatoria actual fueron:

Conjunto	MAE	RMSE	R^2	Error medio
<i>Train</i>	14.1480	29.2039	0.0172	-0,2757
<i>Validation</i>	14.1094	28.4193	0.0191	-0,2473
<i>Test</i>	14.0481	29.1341	0.0181	-0,2175

Cuadro 1.1: Resultados definitivos del modelo manual. El error medio se define como predicción menos valor real y se expresa en minutos.

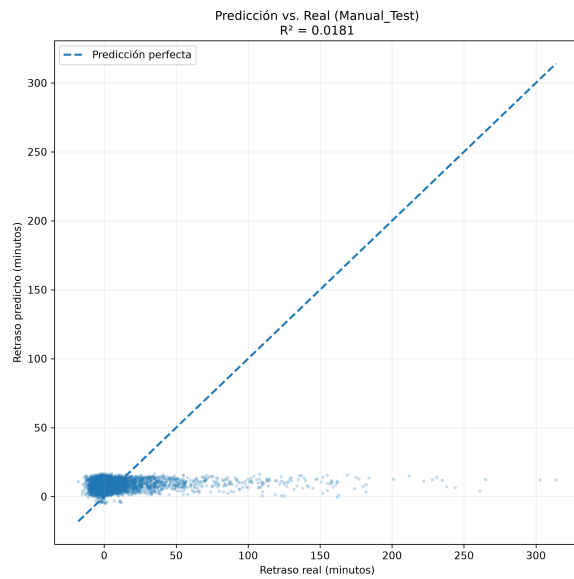
Podemos observar que el RMSE de *train* es mayor que el de *validation*. Esto no significa que el modelo funcione mejor con datos nuevos. La razón es que cada conjunto tiene retrasos extremos diferentes y el RMSE es muy sensible a estos valores. Al comparar *validation* con *test*, el RMSE aumenta 2.51 %, el MAE disminuye 0.43 % y R^2 sólo cambia -0,0010. Como los cambios son

pequeños, podemos decir que el modelo se comporta de forma parecida en ambos conjuntos.

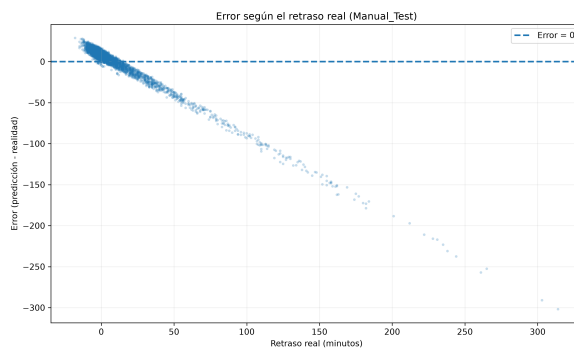
El error absoluto mediano fue de 9.1324 minutos en *validation* y de 9.1173 en *test*. En ambos conjuntos, aproximadamente 23.7 % de las predicciones quedó dentro de ± 5 minutos, 55.3 % dentro de ± 10 , 79.6 % dentro de ± 15 y 92.9 % dentro de ± 30 . Esto quiere decir que el modelo funciona mejor cuando permitimos un margen amplio. La diferencia entre MAE y RMSE también muestra que existen algunos errores muy grandes.

La figura 1.6 contiene dos gráficas sencillas. En la primera, cada punto representa un vuelo: en el eje horizontal aparece su retraso real y en el vertical aparece la predicción. Mientras más cerca se encuentre un punto de la línea punteada, mejor fue la predicción. La segunda gráfica muestra el error de cada vuelo. La línea horizontal representa un error igual a cero; los puntos que están debajo de ella son retrasos que el modelo subestimó.

Estas gráficas sirven para encontrar errores que no se distinguen fácilmente en una tabla. En este caso, muestran que la mayoría de las predicciones se concentra en un rango pequeño y que el modelo predice valores mucho menores cuando el retraso real es muy grande.



(a) Retraso predicho contra retraso real.



(b) Error según el retraso real.

Es importante no confundir este sesgo con el error medio de la tabla. El error medio es simplemente el promedio de la diferencia entre la predicción y el valor real. En *test* fue de $-0,2175$ minutos, por lo que el modelo subestima los retrasos por una cantidad muy pequeña en promedio. Aun así, el sesgo del modelo sigue siendo alto porque su R^2 es muy bajo.

Debido a este *underfitting*, en el siguiente capítulo pruebo XGBoost. Este modelo puede encontrar relaciones más complejas y también permite utilizar las variables categóricas que descarté en la implementación manual.

Figura 1.6: Diagnóstico del modelo manual sobre el conjunto de prueba.

Conclusión sobre sesgo, varianza y ajuste

El modelo tiene un **sesgo alto**, ya que su R^2 se mantiene entre 0.0172 y 0.0191. Esto significa que explica menos de 2 % de los cambios que existen en los retrasos. También tiene una **varianza baja**, porque los resultados de *train*, *validation* y *test* son muy parecidos. Por estas dos razones, podemos concluir que el modelo tiene **underfitting**. En otras palabras, el modelo es demasiado sencillo para explicar correctamente el problema. No se observa *overfitting*, pues el desempeño no empeora de forma importante al utilizar datos que no se usaron para entrenarlo.

Capítulo 2

Modelo con *framework*: XGBoost

2.1. Modificaciones al ETL

XGBoost parte del mismo ETL que expliqué en el primer capítulo. Para no repetir esa información, en esta sección sólo explico los cambios que hice específicamente para este modelo.

A diferencia del modelo manual, para XGBoost sí conservé *UniqueCarrier*, *Origin* y *Dest*. Estas variables pueden ayudar al modelo a encontrar diferencias entre aerolíneas, aeropuertos y rutas. Como son variables categóricas, les apliqué *One-Hot Encoding*. Esta transformación solamente se aplicó a esas tres columnas; las variables de calendario conservaron la codificación cíclica explicada anteriormente.

El codificador se ajustó únicamente con *train*. Después utilicé ese mismo codificador para transformar *validation* y *test*, evitando que estos conjuntos aportaran información durante el entrenamiento. Al terminar este proceso se obtuvo una matriz con **776 predictores**. La matriz se guardó de forma dispersa y con números de 32 bits para reducir el consumo de memoria.

Por último, para XGBoost no utilicé la normalización Min-Max que se aplicó al modelo manual. Al estar formado por árboles, XGBoost realiza divisiones sobre los valores de cada variable y no necesita que todas tengan la misma escala.

Antes de seleccionar XGBoost revisé cinco artículos relacionados con la predicción de retrasos. Sus resultados muestran que los modelos basados en árboles pueden funcionar bien para este problema. Sin embargo, cada trabajo utiliza datos y objetivos diferentes, por lo que no sería correcto comparar directamente sus métricas con las mías. Estos artículos utilizan información del clima, la congestión, la capacidad de los aeropuertos, las operaciones o la propagación de retrasos. Algunos también eliminan valores extremos o trabajan con un solo aeropuerto. Mi *dataset* contiene principalmente información programada del vuelo. Por esta razón, sus resultados no son directamente comparables con los míos: ellos cuentan con variables que explican mejor por qué ocurre un retraso.

2.3. Selección y configuración de XGBoost

Elegí XGBoost porque construye árboles de forma secuencial y cada árbol intenta corregir parte de los errores del anterior. Gracias a esto puede encontrar relaciones no lineales entre los horarios, las fechas, la distancia, la aerolínea y la ruta. Esto es útil porque en el análisis exploratorio observé que la relación lineal con *DepDelay* era muy débil. Además, Manna et al. obtuvieron buenos resultados con Gradient Boosted Decision Tree [5], que pertenece a la misma familia de modelos.

Trabajo	Resultado principal	Diferencia con mi trabajo
Kalliguddi y Leboullec [3]	Regresión: $R^2 = 0,84$ y RMSE de 21.2 min. Random Forest: $R^2 = 0,94$ y RMSE de 12.5 min.	Utiliza variables operacionales que no tengo disponibles.
Rebollo y Balakrishnan [6]	Cerca de 19 % de error de clasificación y 21 min de error mediano a dos horas.	Predice retrasos de rutas y no de vuelos individuales.
Li y Jing [4]	$R^2 = 0,90$, MAE de 5.19 y RMSE de 12.42 min; 93.92 % dentro de ± 15 min.	Utiliza datos de congestión, capacidad y red.
Guo et al. [2]	RFR-MIC funcionó mejor que los modelos comparados. Utilizar varias rutas también mejoró el resultado.	Utiliza información de varias rutas. El resumen no muestra métricas numéricas.
Manna et al. [5]	GBDT obtuvo un R^2 de 92.32 % para llegadas y 94.85 % para salidas.	Es el modelo más parecido a XGBoost, pero su evaluación fue diferente.

Cuadro 2.1: Resultados de los cinco antecedentes revisados. Las métricas pertenecen a conjuntos y objetivos diferentes.

2.3.1. Configuración utilizada

La configuración utilizada para entrenar XGBoost fue la siguiente:

- $n_estimators = 500$
- $max_depth = 6$
- $learning_rate = 0.05$
- $subsample = 0.8$
- $colsample_bytree = 0.8$
- $min_child_weight = 5$
- $reg_lambda = 1.0$
- $tree_method = hist$
- $max_bin = 256$
- $eval_metric = rmse$
- $objective = reg:squarederror$
- $n_jobs = -1$
- $random_state = 42$

El modelo recorrió los 500 árboles. No utilicé detención temprana porque en las pruebas anteriores nunca llegó a detener el entrenamiento y, por lo tanto, no cambió el resultado.

2.3.2. Significado de los hiperparámetros

Cada opción controla una parte diferente del entrenamiento:

$n_estimators$: Es la cantidad de árboles que construye el modelo. En este caso se utilizan 500 árboles.

max_depth : Indica la profundidad máxima de cada árbol. Una profundidad de 6 permite encontrar relaciones más complejas sin hacer árboles demasiado grandes.

$learning_rate$: Controla cuánto aporta cada árbol nuevo a la predicción final. El valor 0.05 hace que el modelo aprenda poco a poco.

$subsample$: Define qué porcentaje de las observaciones utiliza cada árbol. Con 0.8, cada árbol utiliza 80 % de los datos de entrenamiento.

$colsample_bytree$: Define qué porcentaje de los predictores puede utilizar cada árbol. El valor 0.8 significa que utiliza 80 % de las columnas.

min_child_weight : Exige un peso mínimo de 5 para crear una nueva rama. Esto evita divisiones basadas en muy pocos datos.

reg_lambda: Aplica regularización L2 a los valores de las hojas. Sirve para limitar predicciones demasiado extremas.

tree_method: Indica cómo se buscan las divisiones de los árboles. El método *hist* agrupa los valores en intervalos para reducir el tiempo y el uso de memoria.

max_bin: Establece hasta 256 intervalos para el método *hist*. Más intervalos permiten divisiones más detalladas, pero también consumen más recursos.

eval_metric: Define la métrica que se observa durante el entrenamiento. Se utilizó RMSE porque penaliza más los errores grandes.

objective: Indica el tipo de problema y la función que XGBoost intenta minimizar. *reg:squarederror* corresponde a una regresión basada en el error cuadrático.

n_jobs: Controla cuántos núcleos del procesador puede utilizar el entrenamiento. El valor -1 permite utilizar todos los núcleos disponibles.

random_state: Fija la semilla en 42 para que los pasos aleatorios del entrenamiento puedan repetirse.

2.3.3. Técnicas de regularización

Para evitar que XGBoost se ajuste demasiado a *train*, utilicé varias formas de regularización. La profundidad máxima de 6 y *min_child_weight* de 5 limitan qué tan complejos pueden ser los árboles. *Subsample* hace que cada árbol utilice 80 % de las observaciones y *colsample_bytree* hace que utilice 80 % de las columnas. El *learning rate* de 0.05 evita que cada árbol cambie demasiado la predicción. Por último, la regularización L2 limita los valores de las hojas. Con estos parámetros se busca disminuir la varianza y evitar el *overfitting*.

2.4. Resultados de XGBoost

Al igual que con el modelo manual, evalué XGBoost con los conjuntos completos de *train*, *validation* y *test*. Las métricas obtenidas fueron las siguientes:

Conjunto	MAE	RMSE	R^2	Error medio
<i>Train</i>	13.7983	28.6638	0.0532	0.0003
<i>Validation</i>	13.7753	27.9544	0.0509	0.0319
<i>Test</i>	13.7230	28.6896	0.0479	0.0567

Cuadro 2.2: Resultados definitivos de XGBoost. El error medio se expresa en minutos.

Al comparar *validation* con *test*, el RMSE aumenta 2.63 %, el MAE disminuye 0.38 % y R^2 sólo cambia -0.0031 . El error absoluto mediano fue de 8.4836 minutos en *validation* y de 8.5006 en *test*. Como estos valores son muy parecidos, podemos decir que el modelo mantiene su desempeño al utilizar datos nuevos.

En *test*, 27.53 % de las predicciones quedó dentro de ± 5 minutos, 58.50 % dentro de ± 10 , 78.11 % dentro de ± 15 y 92.94 % dentro de ± 30 . Al igual que en el modelo manual, el porcentaje mejora bastante cuando se permite un margen más amplio.

En la figura 2.1 podemos observar cómo disminuye el RMSE conforme se agregan árboles. Las dos líneas continúan bajando hasta el árbol 500 y no se separan cada vez más. Esto apoya la idea de que el modelo no presenta *overfitting* durante estas rondas.

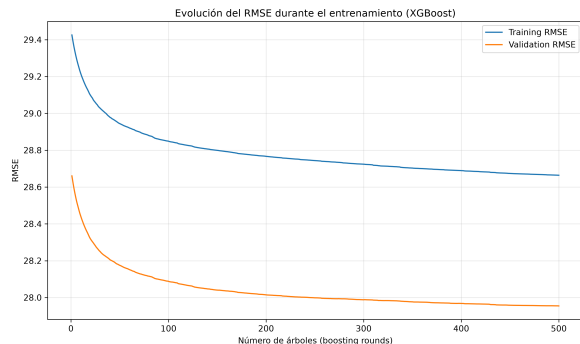
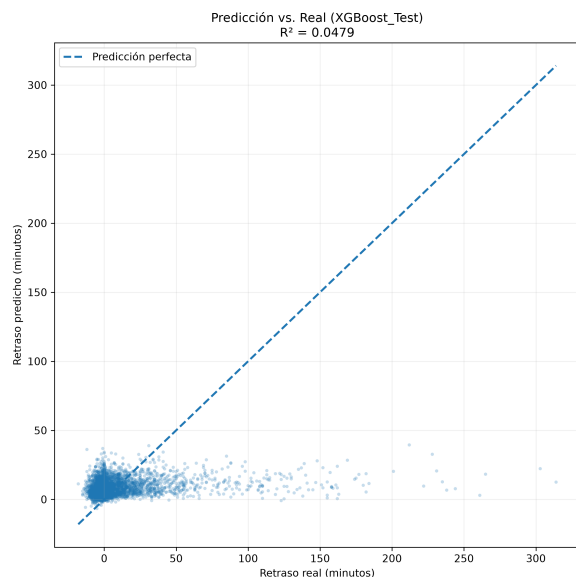
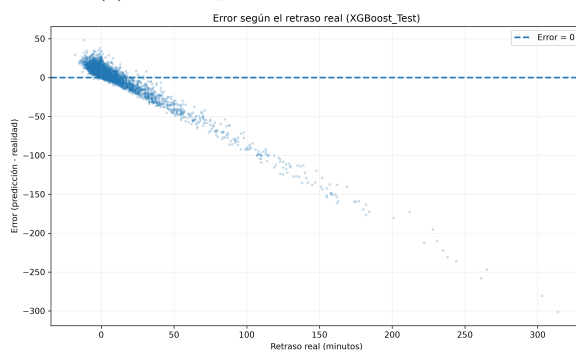


Figura 2.1: Evolución del RMSE durante los 500 árboles de XGBoost.

La figura 2.2 se interpreta de la misma forma que la del modelo manual. La primera gráfica permite revisar qué tan cerca se encuentra cada predicción del valor real. La segunda muestra si el modelo se equivoca por arriba o por debajo del retraso real. En XGBoost las predicciones cubren un rango un poco mayor, pero los retrasos extremos todavía se subestiman.



(a) Retraso predicho contra retraso real.



(b) Error según el retraso real.

Figura 2.2: Diagnóstico de XGBoost sobre el conjunto de prueba.

2.4.1. Conclusión sobre sesgo, varianza y ajuste

XGBoost tiene un **sesgo alto**, aunque es menor que el del modelo manual. Su R^2 en *test* es de 0.0479, por lo que explica aproximadamente 4.8 % de los cambios en los

retrasos. También tiene una **varianza baja**, pues los resultados de *train*, *validation* y *test* son muy parecidos. Por lo tanto, este modelo también tiene **underfitting**. Aunque funciona mejor que el modelo manual, todavía es demasiado limitado para explicar los retrasos extremos con las variables disponibles. No se observa *overfitting*, ya que su desempeño no empeora de forma importante en *validation* o *test*.

El error medio de la tabla fue de +0,0567 minutos en *test*. Esto significa que, en promedio, el modelo casi no sobreestima ni subestima los retrasos. Sin embargo, esto no quiere decir que el sesgo del modelo sea bajo. El R^2 continúa siendo pequeño y las gráficas muestran que los retrasos muy grandes normalmente se subestiman.

2.4.2. Comparación: Manual vs XGBoost

Al comparar ambos modelos, XGBoost disminuye el MAE en 2.31 %, el RMSE en 1.53 % y el error absoluto mediano en 6.76 %. También aumenta R^2 de 0.0181 a 0.0479. En los márgenes de ± 5 y ± 10 minutos, mejora 3.79 y 3.09 puntos porcentuales. El modelo manual obtiene 1.58 puntos más dentro de ± 15 minutos, mientras que dentro de ± 30 minutos prácticamente empatan. Aunque la diferencia no es muy grande, XGBoost obtiene mejores resultados en la mayoría de las métricas y explica una mayor parte de los retrasos.

2.5. Mejora final del modelo XGBoost

Aunque el XGBoost histórico obtuvo mejores resultados que el modelo manual, sus métricas cercanas entre *train* y *validation*, junto con el R^2 bajo, indicaban *underfitting* y no *overfitting*. Como utilizaba varias restricciones de complejidad, primero probé una configuración deliberadamente extrema para comprobar si XGBoost tenía capaci-

Modelo	MAE	RMSE	R^2	MedAE	± 5	± 10	± 15	± 30	Error medio
Manual	14.0481	29.1341	0.0181	9.1173	23.74 %	55.41 %	79.69 %	92.93 %	-0,2175
XGBoost	13.7230	28.6896	0.0479	8.5006	27.53 %	58.50 %	78.11 %	92.94 %	0.0567

Cuadro 2.3: Comparación de los modelos sobre el conjunto de prueba.

dad suficiente para memorizar una muestra al retirar esas restricciones.

Para realizar esta prueba utilicé una fracción pequeña de *train* (0.1 % de los datos) y configuré un modelo con mucha capacidad: 3,000 árboles, profundidad 12 y menos restricciones sobre la creación de ramas. El resultado fue un RMSE de 1.4211 y un R^2 de 0.9972 en *train*, mientras que en *validation* el RMSE aumentó a 31.3369 y el R^2 bajó a -0,1927. Con esto logré provocar *overfitting*: el modelo prácticamente memorizó la muestra de entrenamiento, pero no pudo generalizar a datos que no había utilizado.

Esta configuración extrema no fue una candidata para el modelo final. La prueba confirmó que XGBoost sí tenía capacidad suficiente y que retirar la regularización sin control solamente trasladaba el problema al extremo contrario. A partir de ahí reduje gradualmente la complejidad y volví a ajustar la regularización, utilizando *validation* para comparar las configuraciones. Como resultado de este proceso obtuve una referencia de optimización con 400 árboles, profundidad 7 y *learning rate* de 0.125. Esta referencia fue un resultado intermedio de la búsqueda y no sustituyó al XGBoost histórico. Durante toda la selección mantuve *test* separado y no lo utilicé para comparar configuraciones ni elegir hiperparámetros. Su evaluación se realizó después de congelar la configuración mejorada.

2.5.1. Configuración del modelo mejorado

Además de ajustar los hiperparámetros, modifiqué la representación de las variables temporales. Conservé los componentes seno y coseno, pero los utilicé para reconstruir el mes, el día del mes y el día de la semana

y añadirlos como categorías discretas. También agrupé las horas programadas de salida y llegada en bloques horarios. De esta forma, XGBoost puede separar periodos concretos sin perder la continuidad representada por las variables originales. La codificación *one-hot* se ajustó únicamente con *train*, por lo que ninguna observación de *validation* o *test* influyó en esta preparación. Con las categorías temporales, la representación mejorada alcanzó 874 predictores, frente a los 776 del XGBoost histórico. La tabla 2.4 muestra únicamente los hiperparámetros que modifiqué o añadí durante la mejora.

Parámetro	Histórico 500/6	Mejorado
<i>n_estimators</i>	500	400
<i>learning_rate</i>	0.050	0.125
<i>max_depth</i>	6	0
<i>min_child_weight</i>	5	80
<i>colsample_bytree</i>	0.8	1.0
<i>reg_lambda</i>	1	50
<i>grow_policy</i>	<i>depthwise</i>	<i>lossguide</i>
<i>max_leaves</i>	-	63

Cuadro 2.4: Hiperparámetros modificados o añadidos durante la mejora de XGBoost.

Al reducir la complejidad del experimento extremo observé que las profundidades 5, 6 y 7 mejoraban progresivamente el RMSE de *validation*. Con profundidad 7, la combinación de 400 árboles y un *learning rate* de 0.125 obtuvo un mejor equilibrio que las alternativas con 250 y 300 árboles. Aumenté *min_child_weight* y *reg_lambda* para evitar que el modelo volviera a memorizar la muestra. Las pruebas con valores menores no mejoraron el resultado, por lo que conservé 80 y 50, respectivamente. También cambié *colsample_bytree* de 0.8 a 1.0 para que cada árbol pudiera considerar todos los predictores, incluidas las nuevas

categorías temporales. Aunque al principio retiré restricciones para comprobar la capacidad de XGBoost, estos resultados mostraron que debía recuperarlas para obtener un modelo que generalizara.

El cambio principal fue añadir *grow_policy=lossguide*. Esta política expande primero la hoja que produce la mayor reducción del error, en lugar de hacer crecer todos los nodos de una misma profundidad. Para controlar su complejidad utilicé *max_leaves=63* y dejé *max_depth=0*, de modo que el límite dependiera del número de hojas. Los valores de 127 y 255 hojas redujeron más el error de *train*, pero empeoraron la relación con *validation* y comenzaron a mostrar *overfitting*.

También probé otras representaciones y estructuras. El conjunto completo de variables derivadas aumentó la memoria y empeoró el RMSE; el *target encoding* y los armónicos temporales no produjeron una mejora suficiente; y el modelo residual de dos etapas sólo redujo 0.0013 RMSE mientras casi duplicó el tiempo de ejecución. La combinación de 600 árboles y *learning rate=0.08* obtuvo 0.0011 menos RMSE que la configuración elegida, una diferencia prácticamente nula, pero tardó mucho más y tuvo una variación ligeramente mayor entre semillas. Por esta razón seleccioné el modelo de 400 árboles.

2.5.2. Confirmación de la mejora

La tabla 2.5 resume la confirmación con 50 % de *train* y las semillas de muestreo 42, 123 y 2026. El modelo mejorado ganó en las tres semillas. Su RMSE promedio pasó de 27.8992 a 27.8518, lo que representa una disminución de 0.0474. El MAE y el valor absoluto del error medio también mejoraron. El costo aumentó porque las categorías temporales hicieron más grande la matriz de predictores, pero se mantuvo dentro de un tiempo razonable para este conjunto de

datos.

Métrica	Referencia 400/7	XGBoost mejorado
RMSE	27.8992	27.8518
MAE	13.6978	13.6553
Error medio	0.0212	0.0150
Desviación de RMSE	0.0045	0.0066
Tiempo (s)	59.8611	68.4176

Cuadro 2.5: Confirmación multisevilla en *validation*. La desviación se calcula entre las tres semillas de muestreo.

Como comprobación adicional, entrené ambos XGBoost con todo *train* y conservé *validation* separado. El RMSE pasó de 27.8998 en la referencia 400/7 a 27.8280 en el modelo mejorado, una disminución de 0.0718. La figura 2.3 muestra el seguimiento realizado durante un único entrenamiento. La curva de *train* disminuye conforme se agregan árboles y la de *validation* se estabiliza de manera progresiva. Al final no aparece una separación creciente entre ambas curvas, por lo que no encontré evidencia de que la ganancia fuera resultado de *overfitting*.

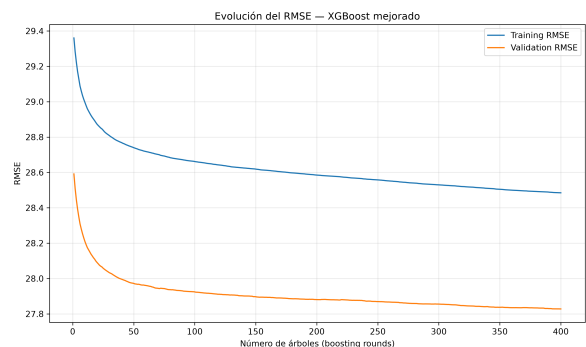


Figura 2.3: Evolución del RMSE de *train* y *validation* durante el entrenamiento del XGBoost mejorado.

2.5.3. Comparación final en *test*

Para la comparación definitiva entrené una sola vez la regresión manual, el XGBoost histórico 500/6 y el XGBoost mejorado. En los dos XGBoost utilicé *validation* únicamente para registrar las curvas del entrenamiento. La ejecución tardó 458.31 segundos, incluyendo el ETL, las gráficas, la preparación, el entrenamiento, la evaluación y

el guardado de los artefactos.

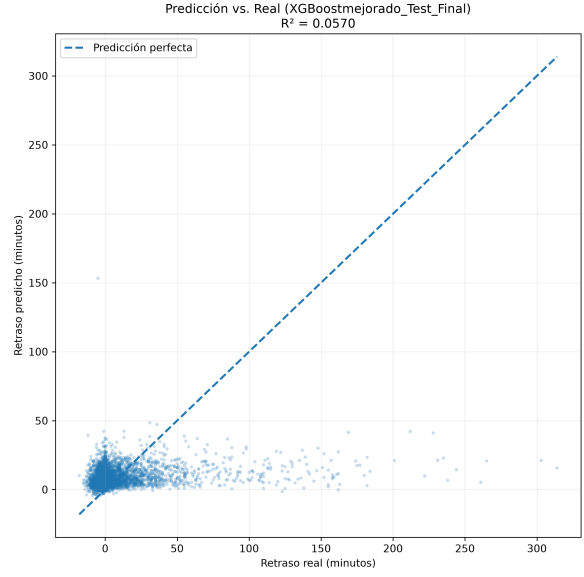
La tabla 2.6 muestra que el XGBoost mejorado obtuvo el menor MAE, RMSE, MedAE y error medio absoluto, además del mayor R^2 . Frente al XGBoost histórico redujo 0.1386 minutos de RMSE y aumentó R^2 de 0.0479 a 0.0570. También aumentó 1.67 puntos porcentuales el porcentaje de predicciones dentro de ± 5 minutos y 1.14 puntos dentro de ± 10 minutos.

Métrica	Manual	XGBoost histórico	XGBoost mejorado
MAE	14.0481	13.7230	13.5822
RMSE	29.1341	28.6896	28.5510
R^2	0.0181	0.0479	0.0570
MedAE	9.1173	8.5006	8.2236
± 5	23.74 %	27.53 %	29.20 %
± 10	55.41 %	58.50 %	59.64 %
± 15	79.69 %	78.11 %	78.12 %
± 30	92.93 %	92.94 %	92.75 %
Error medio	-0.2175	0.0567	0.0450
Tiempo (s)	170.4761	115.1387	111.9381

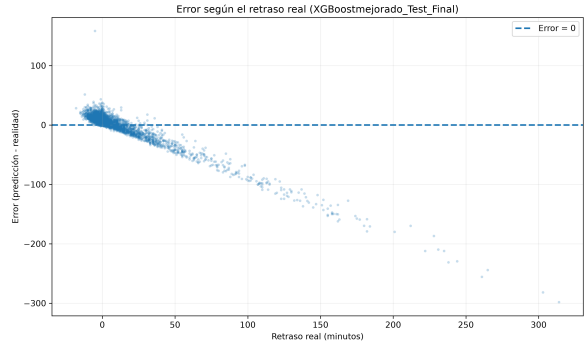
Cuadro 2.6: Comparación final de los tres modelos congelados sobre *test*. El tiempo de cada modelo incluye su preparación, entrenamiento y predicción.

El porcentaje dentro de ± 15 minutos continúa siendo ligeramente mayor para el modelo manual y el XGBoost histórico conserva una diferencia mínima dentro de ± 30 minutos. Sin embargo, el modelo mejorado obtiene el mejor resultado en las métricas principales. En esta ejecución también tardó 3.20 segundos menos que el modelo histórico, aunque una sola medición no permite afirmar que siempre sea más rápido. La ganancia predictiva es pequeña, pero coincide con lo observado durante la optimización y se obtuvo sin incorporar nuevas fuentes de datos.

Para revisar el comportamiento del modelo mejorado utilicé las mismas gráficas diagnósticas de los modelos anteriores. En la figura 2.4, la primera gráfica muestra que la mayoría de las predicciones permanece concentrada cerca de los retrasos más frecuentes, mientras que los valores reales abarcan un intervalo mucho mayor. La segunda gráfica confirma esta situación: los residuos se vuelven más negativos conforme aumenta el retraso real. Por lo tanto, el modelo mejora el error promedio, pero todavía subestima los retrasos extremos.



(a) Retraso predicho contra retraso real.



(b) Error según el retraso real.

Figura 2.4: Diagnóstico del XGBoost mejorado sobre el conjunto de prueba.

La tabla 2.7 muestra diez ejemplos de la misma ejecución. En la segunda observación, cuyo retraso real fue de 84 minutos, el XGBoost mejorado se acerca más que los otros modelos, pero sólo predice 16.2088 minutos. Este caso coincide con la subestimación observada en las gráficas.

Observación	Real	Manual	XGBoost histórico	XGBoost mejorado
1	10.0000	6.8353	8.2458	8.5727
2	84.0000	13.6229	14.5761	16.2088
3	-4.0000	5.7213	4.2166	6.8110
4	-4.0000	9.9137	11.3881	12.2500
5	1.0000	13.8972	19.3571	17.6545
6	2.0000	5.5418	4.5724	5.2031
7	12.0000	13.7533	15.2334	13.8534
8	31.0000	4.2279	4.2563	5.8151
9	-1.0000	11.6341	4.1842	5.1145
10	-1.0000	5.7292	13.3906	12.4339

Cuadro 2.7: Primeras diez predicciones de la comparación final sobre *test*.

Por último, en la figura 2.5 agrupé los vuelos para comparar el promedio real con el promedio predicho de los tres modelos. Para cada modelo ordené por separado sus predicciones y formé 20 grupos de tamaño parecido; por esta razón, los puntos de las tres curvas no contienen necesariamente los mismos vuelos. Los dos XGBoost cubren un rango de predicciones mayor que el modelo manual, pero los tres se apartan considerablemente de la línea de predicción perfecta en el grupo con mayor retraso estimado.

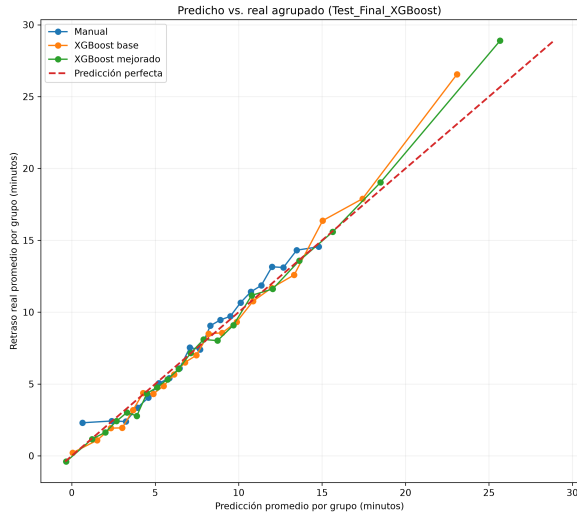


Figura 2.5: Promedios predichos y reales en *test* para los tres modelos.

2.5.4. Conclusión sobre sesgo, varianza y ajuste

El XGBoost mejorado mantiene un **sesgo alto**, aunque es menor que el de los dos modelos anteriores. Su R^2 en *test* es de 0.0570, por lo que explica aproximadamente 5.7 % de los cambios en los retrasos. También tiene una **varianza baja**, pues los resultados de *train*, *validation* y *test* se mantienen cercanos. Por lo tanto, el modelo todavía presenta **underfitting**. No se observa *overfitting* en la configuración final, ya que las curvas de aprendizaje no se separan de forma creciente y el desempeño no empeora fuera de *train*.

El error medio fue de +0,0450 minutos en

test, por lo que el modelo casi no sobreestima ni subestima los retrasos en promedio. Este valor no significa que su sesgo sea bajo: el R^2 continúa siendo pequeño y las gráficas muestran que los retrasos extremos generalmente se subestiman.

2.6. Discusión final

2.6.1. Por qué los modelos no fueron suficientes

El XGBoost mejorado obtuvo los mejores resultados generales, pero no fue suficientemente preciso para predecir el retraso de un vuelo individual. En *test* alcanzó un MAE de 13.5822 minutos, un RMSE de 28.5510 minutos y un R^2 de 0.0570. Esto significa que sólo explicó cerca de 5.7 % de los cambios observados en los retrasos. Frente al XGBoost histórico, la mejora fue consistente, pero pequeña: el MAE disminuyó 0.1408 minutos y el RMSE 0.1386 minutos.

El error medio de +0,0450 minutos tampoco implica que las predicciones individuales sean cercanas a los valores reales. Ese promedio indica que las sobreestimaciones y las subestimaciones casi se compensan entre sí, aunque pueden existir errores grandes en vuelos concretos. De forma similar, que 92.75 % de las predicciones se encuentre dentro de ± 30 minutos representa un margen amplio y no contradice el R^2 bajo ni la subestimación de los retrasos extremos.

La distribución de la variable objetivo ayuda a explicar esta dificultad. Aproximadamente 9.10 % de los vuelos tiene un retraso absoluto mayor de 30 minutos y ese grupo concentra cerca de 89 % de las desviaciones cuadráticas respecto de la media. Por lo tanto, una fracción relativamente pequeña de observaciones domina gran parte de la variabilidad que los modelos deben explicar. Las variables programadas permiten reconocer tendencias generales, pero contienen poca información sobre los sucesos que originan los retrasos grandes.

2.6.2. Posibles mejoras

Una mejora posterior podría incorporar información operacional, como el clima, la congestión, la capacidad de los aeropuertos, la disponibilidad o rotación de las aeronaves y los retrasos previos. Estas variables describen condiciones que pueden originar o propagar un retraso y que no están presentes en el conjunto utilizado.

También sería conveniente recuperar el año y una fecha completa para ordenar cronológicamente los vuelos. Esto permitiría entrenar con periodos anteriores y evaluar con periodos posteriores, en lugar de limitar el análisis a una división aleatoria de la misma población histórica.

Por último, sería necesario revisar los valores extremos y establecer una política explícita para distinguir observaciones reales de posibles errores de captura. Estos registros no deberían eliminarse automáticamente, pues los retrasos grandes forman parte del fenómeno estudiado.

2.6.3. Conclusiones finales

El trabajo permitió construir y refinar el proceso de ETL, implementar una regresión lineal mediante Descenso de Gradiente, comparar esa implementación con XGBoost y realizar una optimización sistemática del modelo con *framework*. XGBoost capturó relaciones adicionales y mejoró de manera consistente las métricas del modelo manual, mientras que la configuración final obtuvo una ganancia adicional sobre el XGBoost histórico.

Sin embargo, las mejoras no fueron suficientes para resolver el problema predictivo con las variables disponibles. Los tres modelos mantuvieron un sesgo alto y explicaron una proporción pequeña de la variación de los retrasos. En consecuencia, la principal limitación final parece encontrarse en la información de entrada, más que en la capacidad técnica del algoritmo.

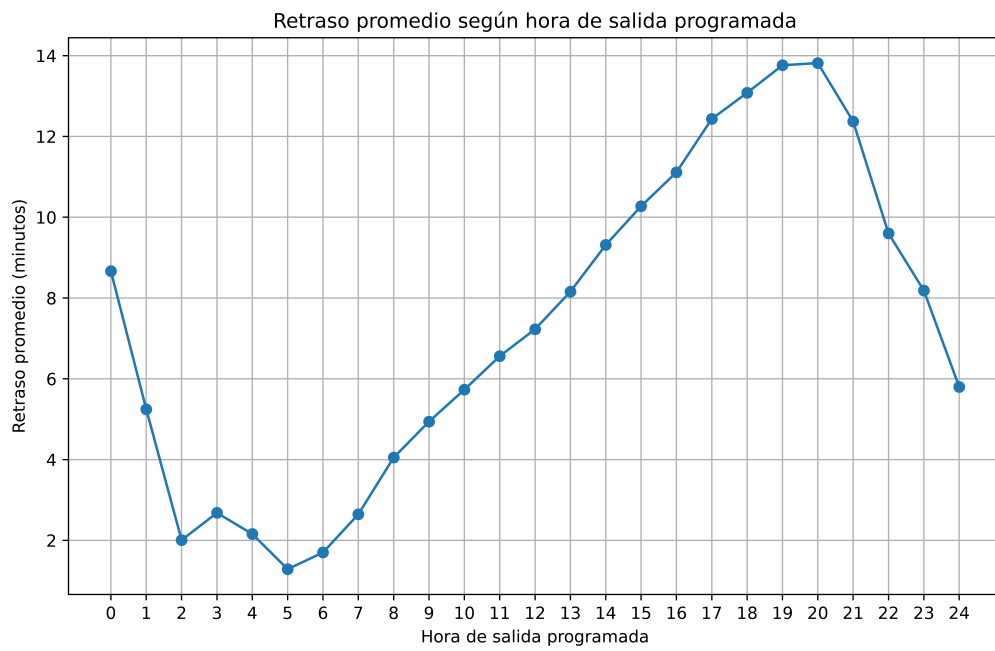
Bibliografía

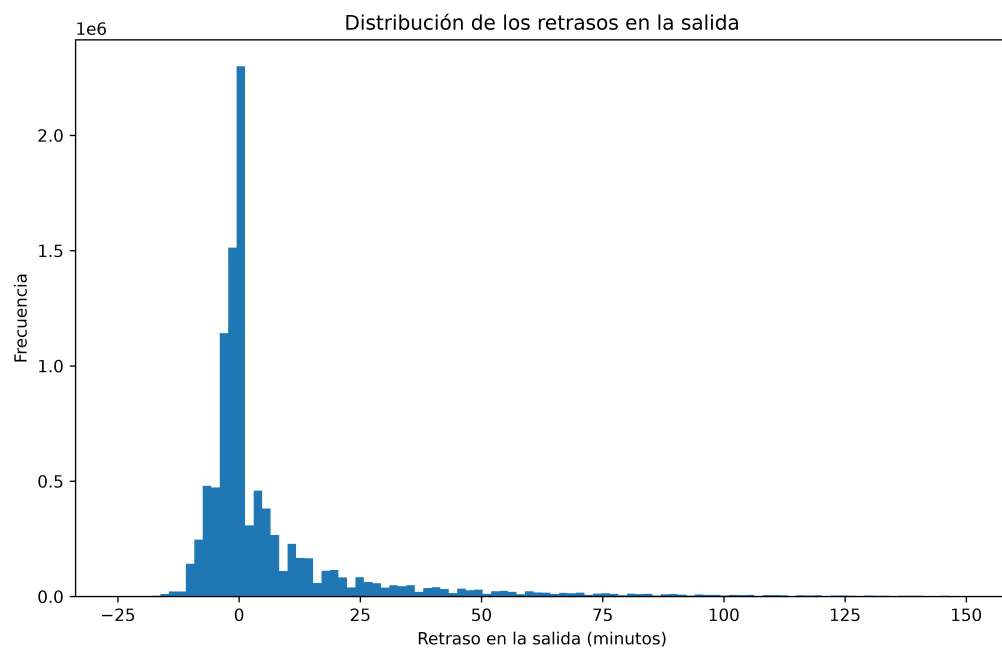
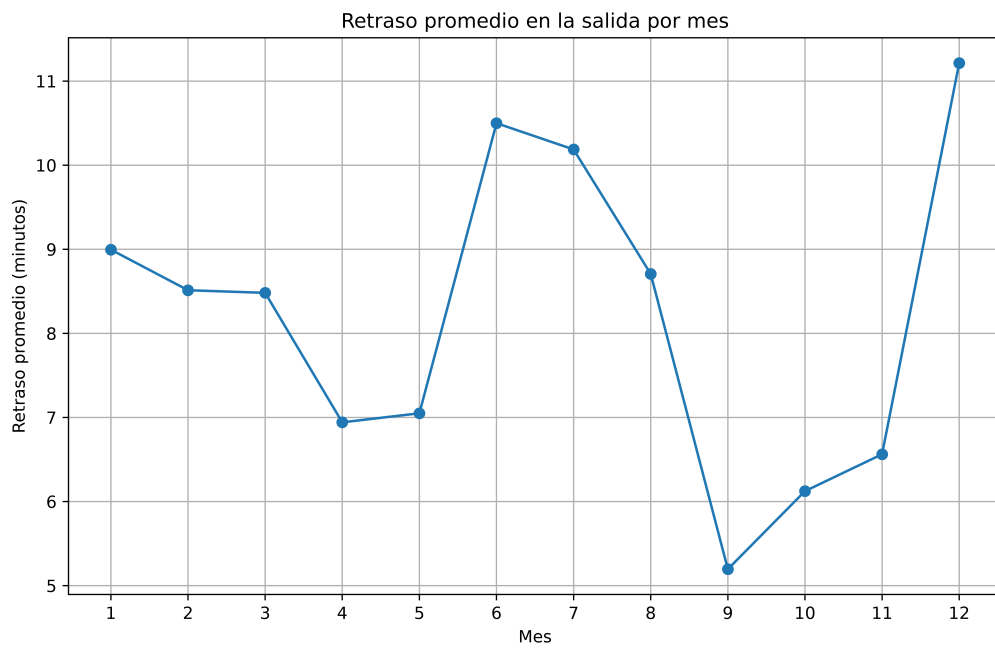
- [1] Matthew Kosinski Annie Badman. *¿Qué es Apache Parquet? / IBM* — *ibm.com*. <https://www.ibm.com/mx-es/think/topics/parquet>. [Accessed 26-08-2026].
- [2] Zhen Guo et al. “A Novel Hybrid Method for Flight Departure Delay Prediction Using Random Forest Regression and Maximal Information Coefficient”. En: *Aerospace Science and Technology* 116 (2021), pág. 106822. DOI: [10.1016/j.ast.2021.106822](https://doi.org/10.1016/j.ast.2021.106822).
- [3] Anish M. Kalliguddi y Aera K. Leboulluec. “Predictive Modeling of Aircraft Flight Delay”. En: *Universal Journal of Management* 5.10 (2017), págs. 485-491. DOI: [10.13189/ujm.2017.051003](https://doi.org/10.13189/ujm.2017.051003).
- [4] Qiang Li y Ranzhe Jing. “Generation and Prediction of Flight Delays in Air Transport”. En: *IET Intelligent Transport Systems* 15.6 (2021), págs. 740-753. DOI: [10.1049/itr2.12057](https://doi.org/10.1049/itr2.12057).
- [5] Suvojit Manna et al. “A Statistical Approach to Predict Flight Delay Using Gradient Boosted Decision Tree”. En: *2017 International Conference on Computational Intelligence in Data Science (ICCIDS)*. 2017. DOI: [10.1109/ICCIDS.2017.8272656](https://doi.org/10.1109/ICCIDS.2017.8272656).
- [6] Juan Jose Rebollo y Hamsa Balakrishnan. “Characterization and Prediction of Air Traffic Delays”. En: *Transportation Research Part C: Emerging Technologies* 44 (2014), págs. 231-241. DOI: [10.1016/j.trc.2014.04.007](https://doi.org/10.1016/j.trc.2014.04.007).
- [7] scgupta. *Training, Validation, and Test Split for ML Models* — *ml4devs.com*. <https://www.ml4devs.com/what-is/train-validation-test-data-split/>. [Accessed 26-08-2026].

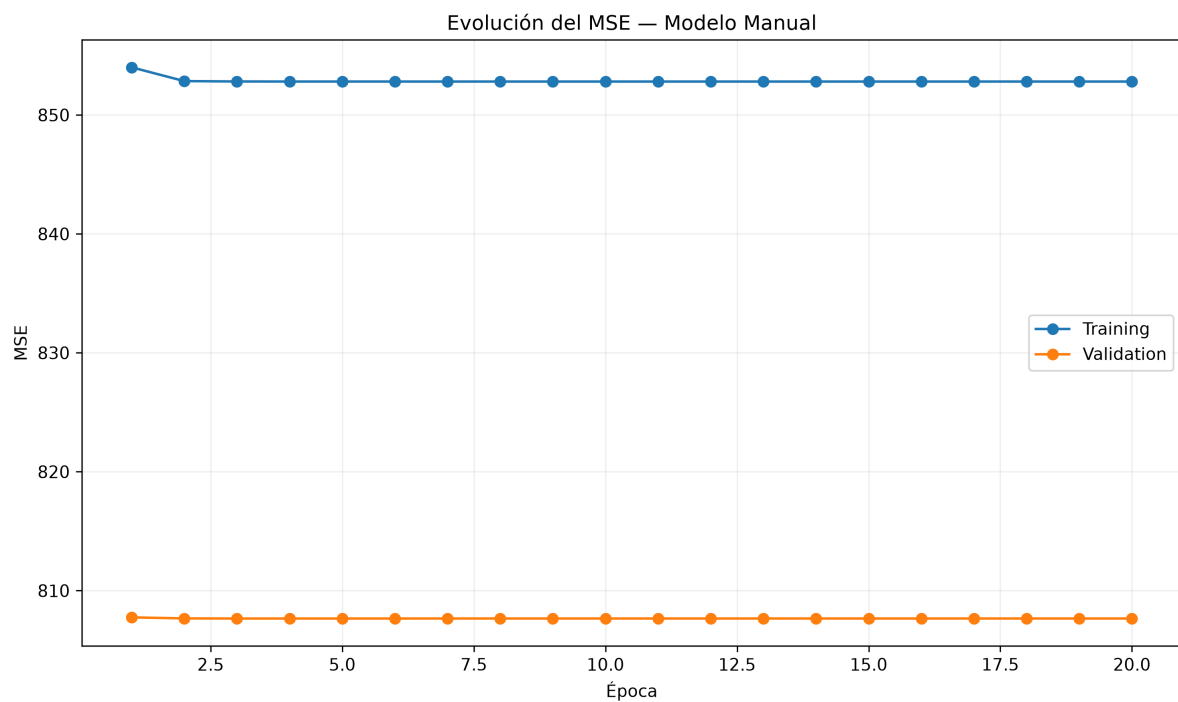
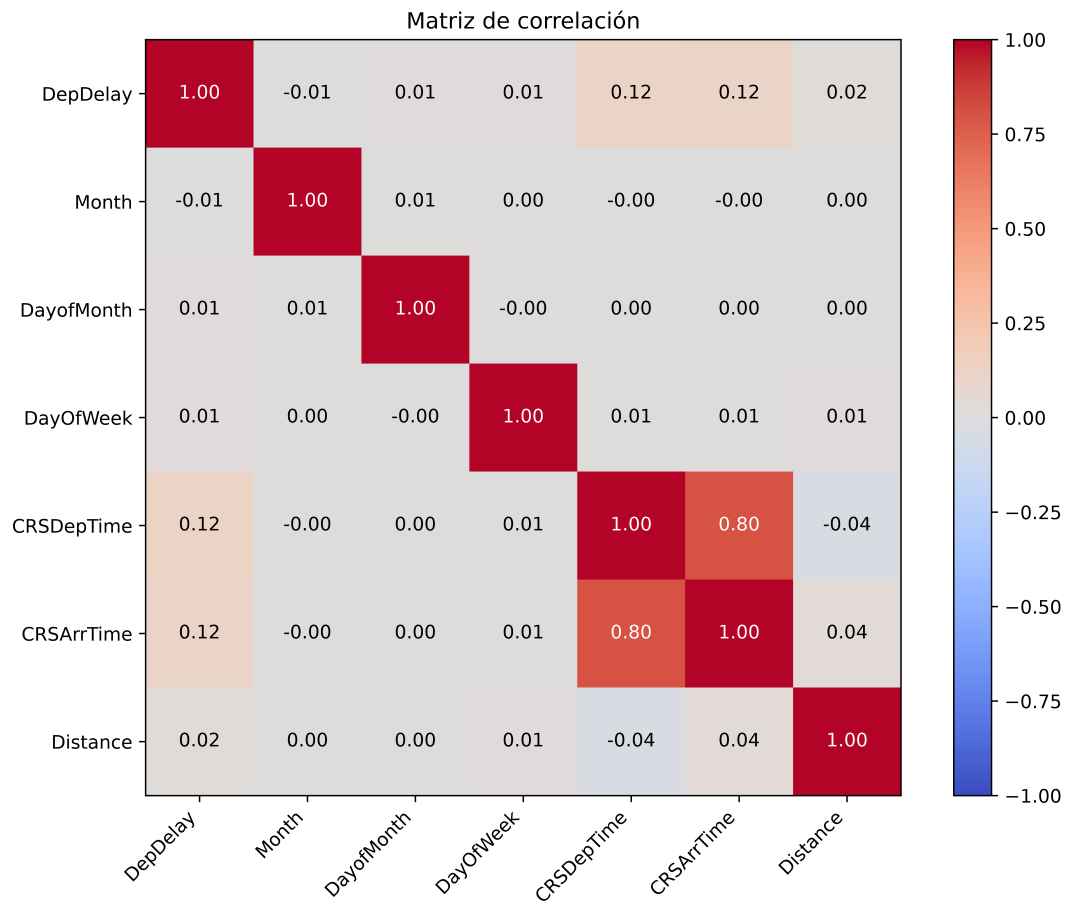
Apêndice A

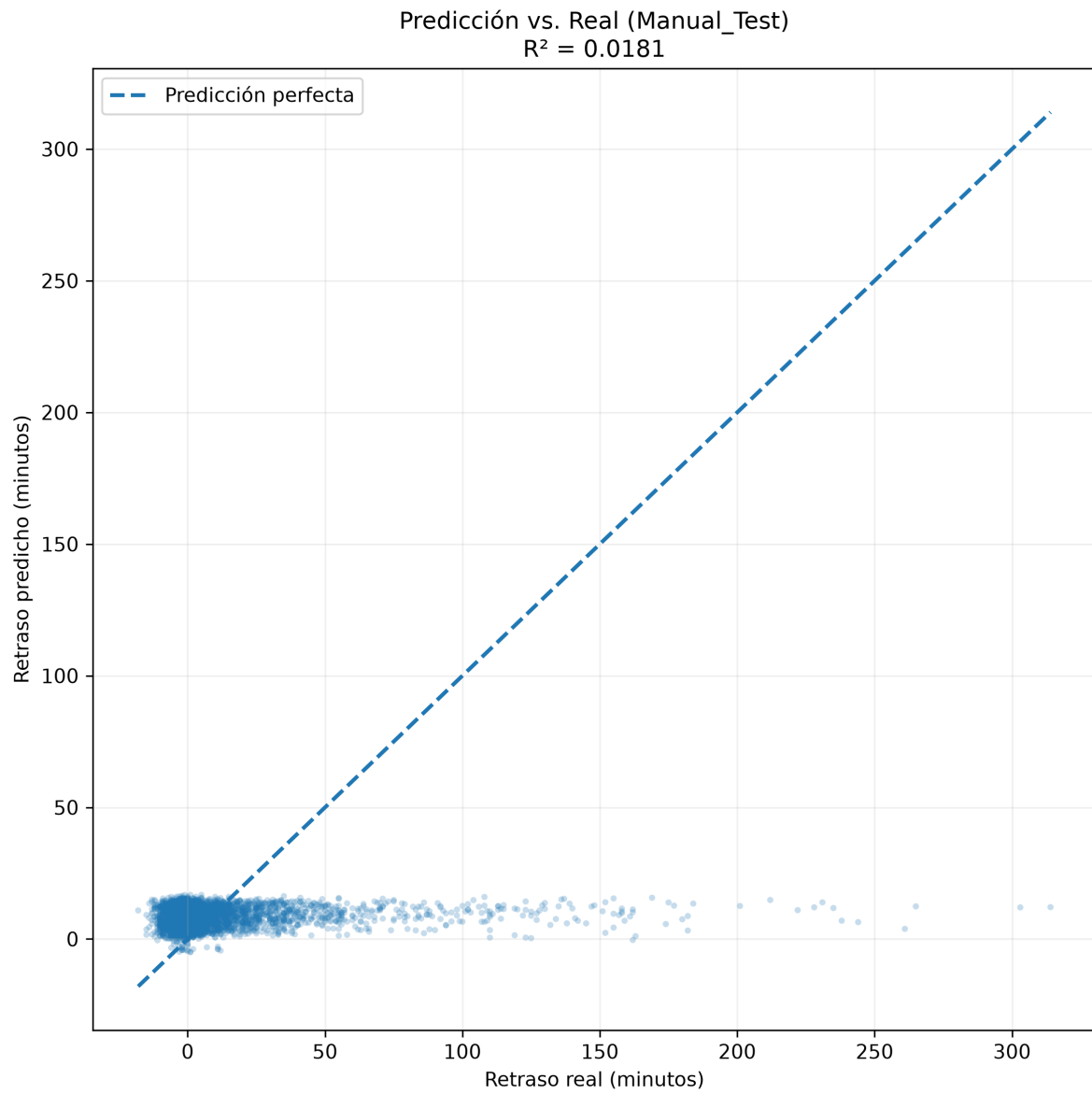
Figuras ampliadas

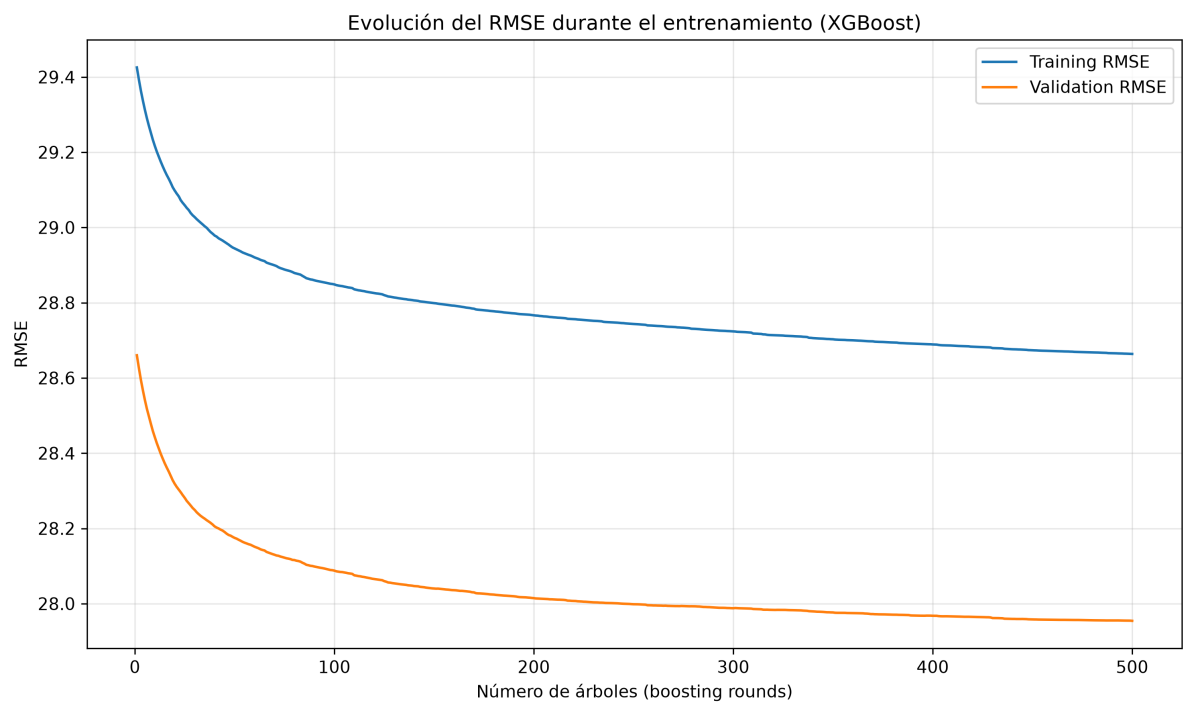
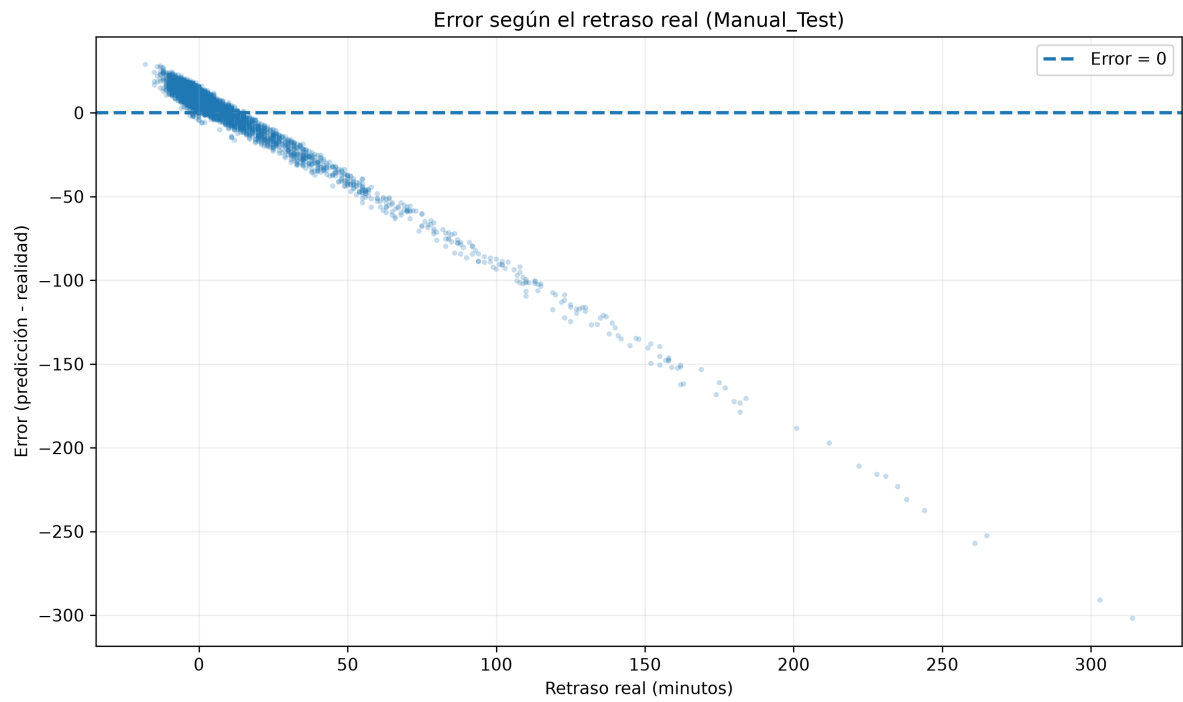


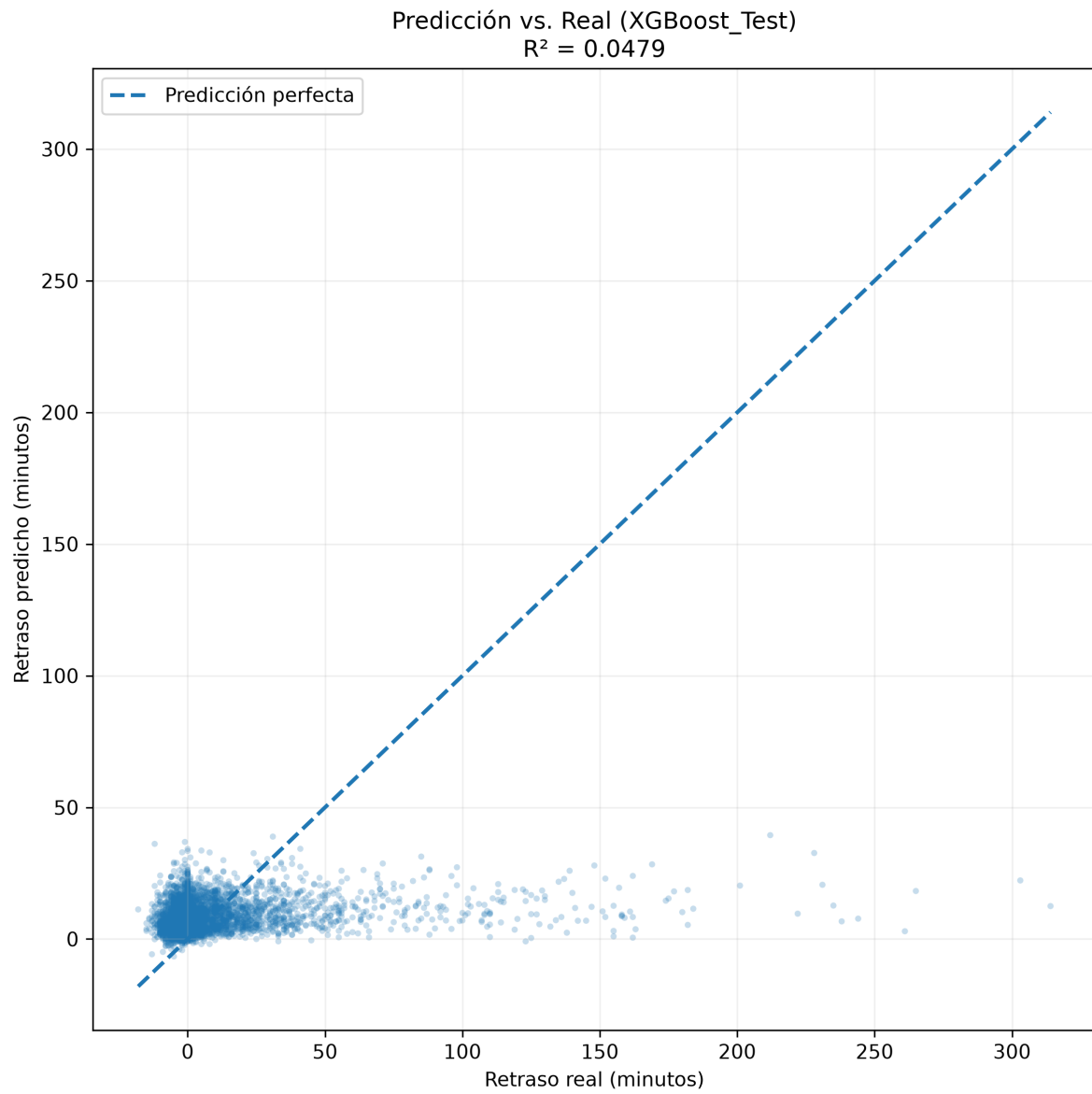


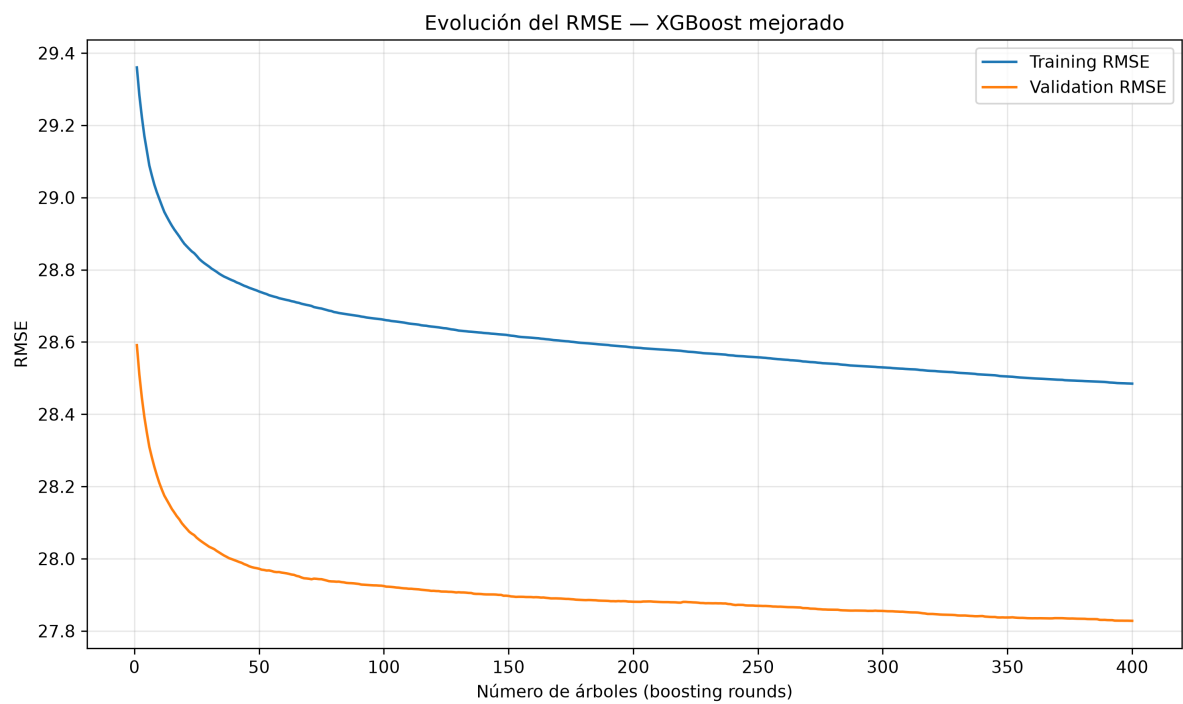
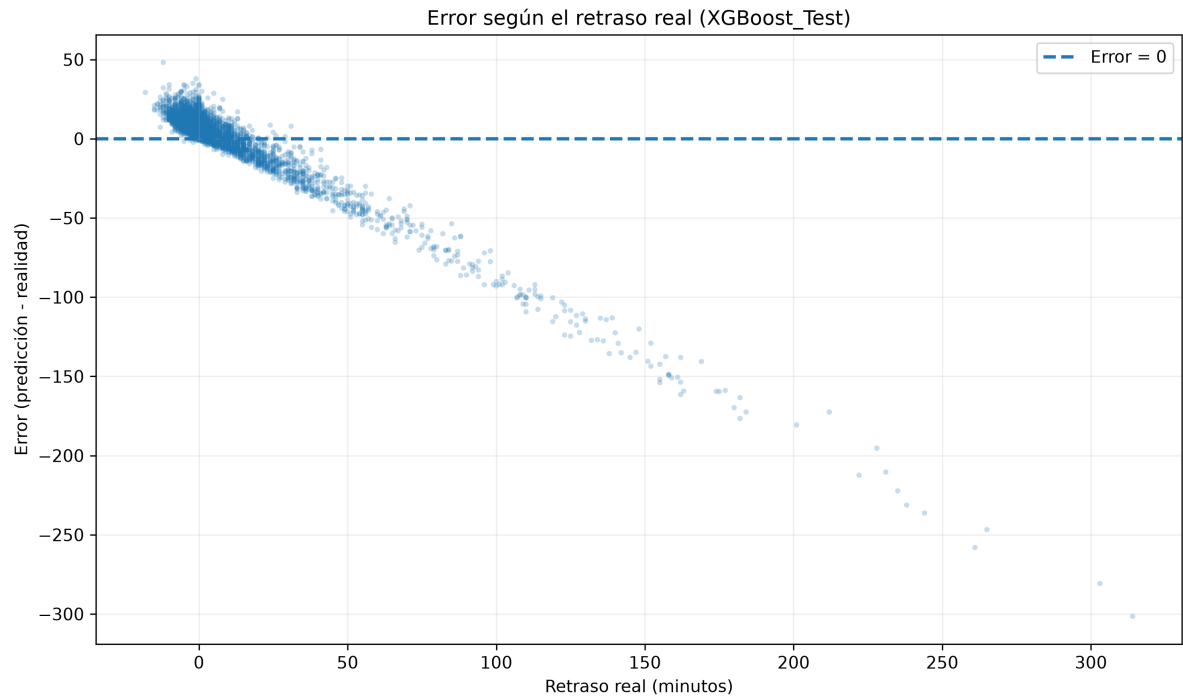


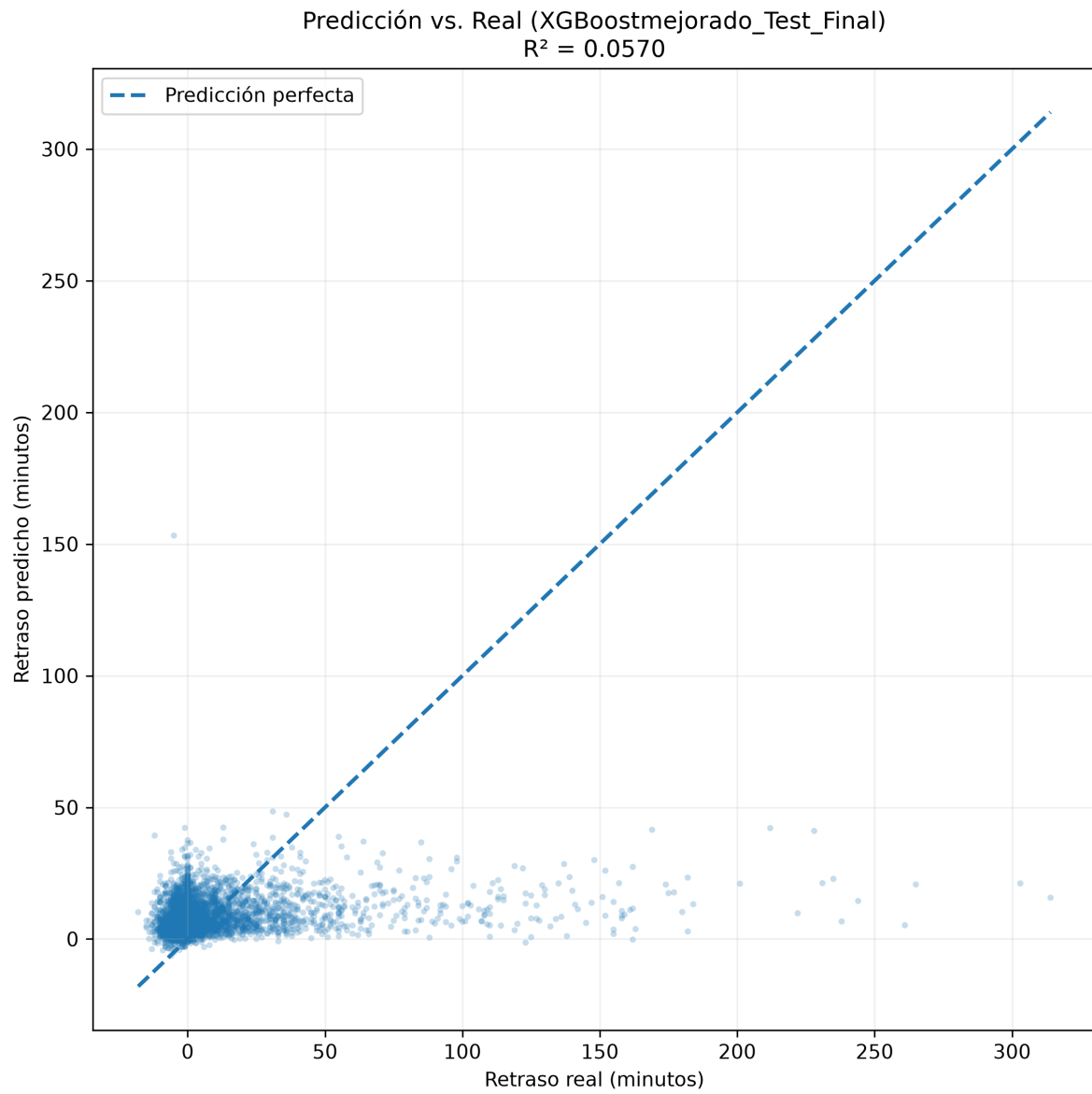


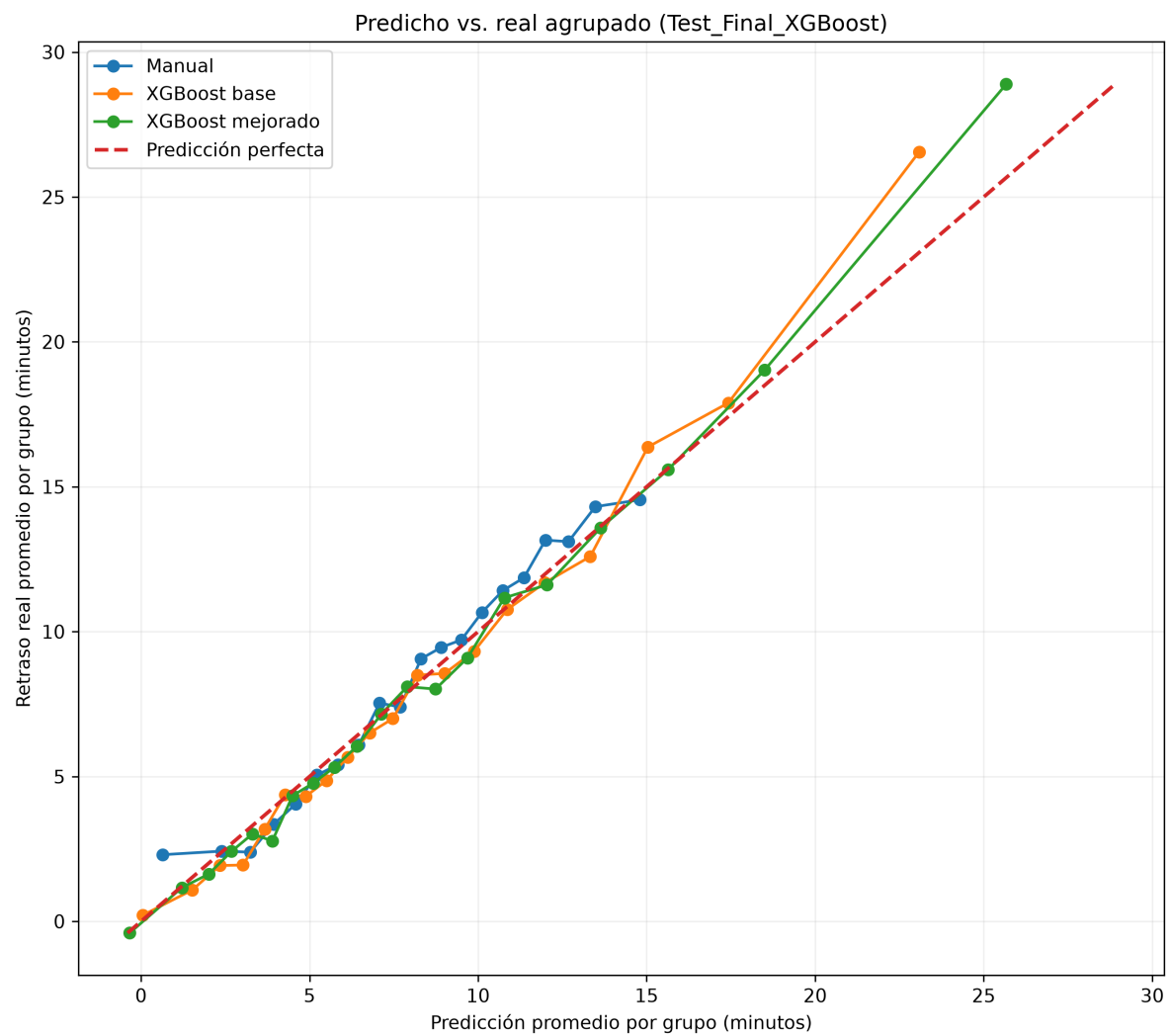
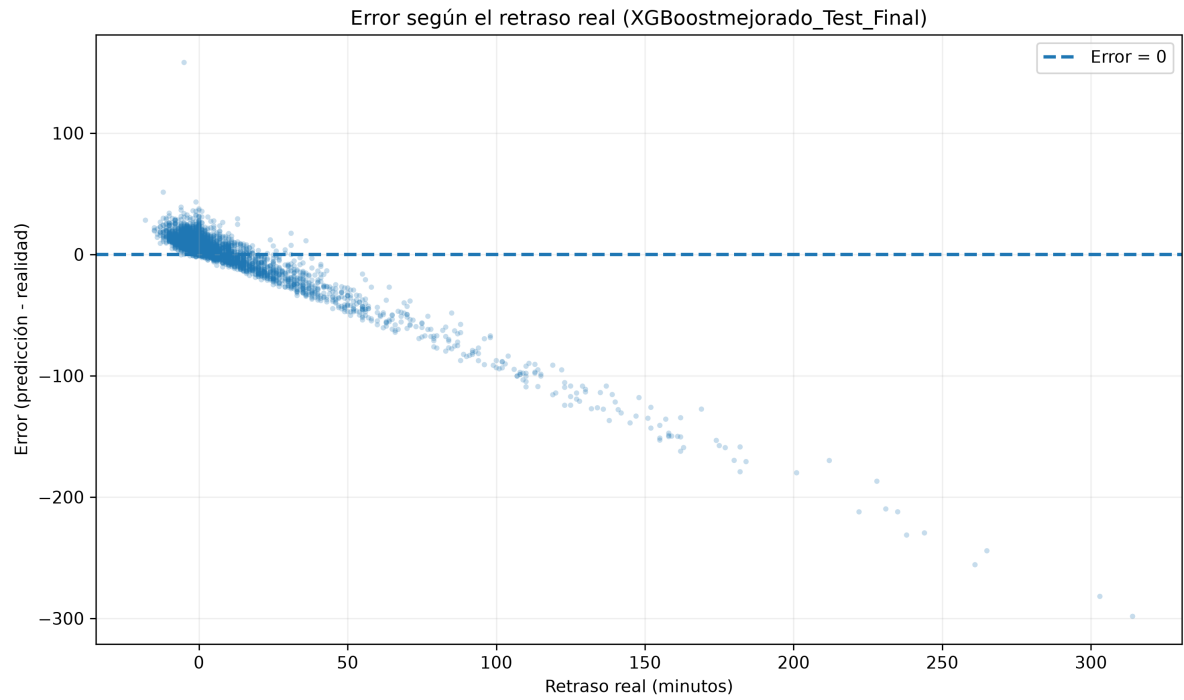












Apéndice B

Códigos

Para consultar todos los códigos utilizados en este reporte, viste: <https://github.com/TheSimpleMau/Concentracion-IA-Portafolio-de-Implementacion>. Aquí sólo se dejarán los tres principales códigos que implementan los modelos y los ejecutan.

Implementación del modelo de regresión lineal.

```
1 import math
2 import random
3 from tqdm import tqdm # Para crear una barra de progreso al
  momento de entrenar el modelo
4
5 def hypo(params: list, weights: list, bias: float) -> float:
6     result = 0
7     for i in range(len(params)):
8         result += params[i] * weights[i]
9     return result + bias
10
11 def error(y_pred:float,y_real:float)->float:
12     return y_pred - y_real
13
14 def cost(y_hat:list, y:list)->float:
15     result = 0
16     for i in range(len(y)):
17         result += (error(y_hat[i],y[i]))**2
18     result = 1/len(y) * result
19     return result
20
21 def r_squared(y_pred: list, y_real: list) -> float:
22     n = len(y_real)
23     mean_y = sum(y_real) / n
24     ss_res = 0.0
25     ss_tot = 0.0
26     for i in range(n):
27         ss_res += (y_real[i] - y_pred[i]) ** 2
28         ss_tot += (y_real[i] - mean_y) ** 2
29     return 1 - (ss_res / ss_tot)
30
31 def gradient(weights: list, bias: float, params: list, y: list):
32     gradients = [0] * len(weights)
```

```

33     bias_gradient = 0
34     m = len(params)
35
36     for i in range(m):
37         predict = hypo(params[i], weights, bias)
38         predict_error = error(predict, y[i])
39
40         for j in range(len(weights)):
41             gradients[j] += predict_error * params[i][j]
42         bias_gradient += predict_error
43
44     for j in range(len(weights)):
45         gradients[j] = (2/m) * gradients[j]
46     bias_gradient = (2/m) * bias_gradient
47
48     return gradients, bias_gradient
49
50
51 def update(weights:list, bias:float, gradients:list, bias_gradient
52 :float, lr:float):
53     new_weights = []
54     for i in range(len(weights)):
55         new_weight = weights[i] - (lr * gradients[i])
56         new_weights.append(new_weight)
57     new_bias = bias - (lr * bias_gradient)
58     return new_weights, new_bias
59
60 def predict(params:list, weights:list, bias:float):
61     predictions = []
62     for i in range(len(params)):
63         prediction = hypo(params[i], weights, bias)
64         predictions.append(prediction)
65     return predictions
66
67
68 def train_step(weights:list, bias:float, params:list, y:list, lr:
69 float):
70     predictions = predict(params, weights, bias)
71     current_cost = cost(predictions, y)
72     gradients, bias_gradient = gradient(weights, bias, params, y)
73     new_weights, new_bias = update(weights, bias, gradients,
74                                     bias_gradient, lr)
75     return new_weights, new_bias, current_cost
76
77 def train(params:list, y:list, validation_params:list,
78 validation_y:list, lr:float, epochs:int, batch_size:int,
79 random_state:int=42):
80     # weights = [0 for _ in range(len(params[0]))]
81     rng = random.Random(random_state)
82     weights = [rng.random() for _ in range(len(params[0]))]

```

```

79     bias = 0
80     train_costs = []
81     validation_costs = []
82
83     m = len(params)
84     num_batches = math.ceil(m / batch_size)
85
86     total_steps = epochs*num_batches
87     pbar = tqdm(total=total_steps, desc="Entrenando modelo")
88
89     for epoch in range(epochs):
90         epoch_train_cost = 0
91         # Iteramos sobre los datos en pequeños lotes
92         for i in range(num_batches):
93             # Calculamos los índices de inicio y fin
94             start_idx = i * batch_size
95             end_idx = min(start_idx + batch_size, m)
96             # Extraemos el lote actual
97             batch_params = params[start_idx:end_idx]
98             batch_y = y[start_idx:end_idx]
99             # Entrenamos sólo con este lote.
100             weights, bias, batch_cost = train_step(weights, bias,
101                                                         batch_params, batch_y, lr)
102             # Sumamos el costo del lote para sacar el promedio
103             # después
104             epoch_train_cost += batch_cost * len(batch_y)
105             # Actualizar barra de progreso
106             pbar.update(1)
107             pbar.set_postfix({
108                 "ep": f"{epoch+1}/{epochs}",
109                 "batch": f"{i+1}/{num_batches}",
110             })
111
112             # Costo promedio de entrenamiento de toda la época
113             epoch_train_cost /= m
114             train_costs.append(epoch_train_cost)
115
116             # Evaluamos en validación
117             validation_predictions = predict(validation_params,
118                                             weights, bias)
119             validation_cost = cost(validation_predictions,
120                                   validation_y)
121             validation_costs.append(validation_cost)
122
123     pbar.close()
124     return weights, bias, train_costs, validation_costs
125
126 def run_regression_model(X_train, y_train, X_val, y_val, X_test=None,
127                         y_test=None, learning_rate=0.1, epochs=20, batch_size=2048,
128                         random_state=42):

```

```

124     params = X_train.values.tolist()
125     train_y = y_train.tolist()
126     validation_params = X_val.values.tolist()
127     validation_y = y_val.tolist()
128     if (X_test is None) != (y_test is None):
129         raise ValueError("X_test y y_test deben proporcionarse
130                             juntos.")
131
132     weights, bias, train_costs, validation_costs = train(params,
133     train_y, validation_params, validation_y, learning_rate,
134     epochs, batch_size, random_state)
135
136     train_predictions = predict(params, weights, bias)
137     validation_predictions = predict(validation_params, weights,
138     bias)
139     predictions = {
140         "train": train_predictions,
141         "val": validation_predictions,
142         "y_train": train_y,
143         "y_val": validation_y,
144     }
145
146     if X_test is not None:
147         test_params = X_test.values.tolist()
148         test_y = y_test.tolist()
149         predictions["test"] = predict(test_params, weights, bias)
150         predictions["y_test"] = test_y
151
152     history = {
153         "epoch": list(range(1, len(train_costs) + 1)),
154         "train_mse": train_costs,
155         "val_mse": validation_costs,
156         "train_mse_kind": "online_during_updates"
157     }
158
159     model = {
160         "type": "manual_linear_regression",
161         "weights": weights,
162         "bias": bias,
163         "learning_rate": learning_rate,
164         "epochs": epochs,
165         "batch_size": batch_size,
166         "random_state": random_state
167     }
168
169     return model, predictions, history

```

Modelo XGBoost (base y mejorado).

```

1 from xgboost import XGBRegressor
2
3

```

```

4 def train_xgboost(X_train, y_train, X_val, y_val, n_estimators
=1000, **xgb_kwargs):
5     params = {
6         "n_estimators": n_estimators,
7         "max_depth": 8,
8         "learning_rate": 0.05,
9         "subsample": 0.8,
10        "colsample_bytree": 0.8,
11        "min_child_weight": 5,
12        "tree_method": "hist",
13        "eval_metric": "rmse",
14        "objective": "reg:squarederror",
15        "n_jobs": -1,
16        "random_state": 42
17    }
18    params.update(xgb_kwargs)
19
20    model = XGBRegressor(**params)
21    model.fit(X_train, y_train, eval_set=[ (X_train, y_train), (
        X_val, y_val)], verbose=100)
22    results = model.evals_result()
23    history = {
24        "iteration": list(range(1, len(results["validation_0"]["
            rmse"]) + 1)),
25        "train_rmse": results["validation_0"]["rmse"],
26        "val_rmse": results["validation_1"]["rmse"],
27    }
28
29    return model, history
30
31
32 def predict(model, X_train, X_val, X_test=None):
33     predictions = {
34         "train": model.predict(X_train),
35         "val": model.predict(X_val),
36     }
37     if X_test is not None:
38         predictions["test"] = model.predict(X_test)
39     return predictions
40
41
42 def run_xgboost(X_train, y_train, X_val, y_val, X_test=None,
y_test=None, n_estimators=1000, **xgb_kwargs):
43     if (X_test is None) != (y_test is None):
44         raise ValueError("X_test y y_test deben proporcionarse
            juntos.")
45
46     model, history = train_xgboost(
47         X_train,
48         y_train,
49         X_val,

```

```
50     y_val,
51     n_estimators=n_estimators,
52     **xgb_kwargs,
53 )
54
55 predictions = predict(model, X_train, X_val, X_test)
56 predictions.update({
57     "y_train": y_train.tolist(),
58     "y_val": y_val.tolist(),
59 })
60 if y_test is not None:
61     predictions["y_test"] = y_test.tolist()
62
63 return model, predictions, history
```

Para gestionar la ejecución de los tres modelos.

```
1 import json
2 import pickle
3 import time
4 from datetime import datetime
5 from pathlib import Path
6
7 import pandas as pd
8
9 from etlProcess import extract_data, transform_data, split_data
10 from preprocessing import prepare_model_data
11 from regressionModel import run_regression_model
12 from xgboostModel import run_xgboost
13 from evaluation import (
14     create_comparison_dataframe,
15     create_diagnostic_dataframe,
16     evaluate_all_splits,
17     print_diagnostic_summary,
18 )
19 from experimentStorage import save_experiment
20 from plotUtil import (
21     binned_prediction_plot,
22     compare_models_bar,
23     correlation_matrix,
24     delays_hist,
25     error_distribution,
26     mean_delay_by_hour,
27     mean_delay_by_month,
28     mean_delay_by_week,
29     plot_training_history,
30     predicted_vs_actual,
31     residual_plot,
32     xgb_cost_evolution,
33 )
34
35
```

```
36 FINAL_COMPARISON_DIRECTORY = Path("resultados/
    evaluacion_final_xgboost")
37 FINAL_REQUIRED_VALUES = {
38     "sample_fraction": 1.0,
39     "shuffle_data": True,
40     "random_state": 42,
41 }
42 MANUAL_PARAMETERS = {
43     "learning_rate": 0.01,
44     "epochs": 20,
45     "batch_size": 2048,
46     "random_state": 42,
47 }
48 XGBOOST_BASE_PARAMETERS = {
49     "n_estimators": 500,
50     "max_depth": 6,
51     "learning_rate": 0.05,
52     "subsample": 0.8,
53     "colsample_bytree": 0.8,
54     "min_child_weight": 5,
55     "reg_lambda": 1.0,
56     "tree_method": "hist",
57     "max_bin": 256,
58     "eval_metric": "rmse",
59     "objective": "reg:squarederror",
60     "n_jobs": -1,
61     "random_state": 42,
62 }
63 XGBOOST_IMPROVED_PARAMETERS = {
64     "n_estimators": 400,
65     "max_depth": 0,
66     "max_leaves": 63,
67     "grow_policy": "lossguide",
68     "learning_rate": 0.125,
69     "subsample": 0.8,
70     "colsample_bytree": 1.0,
71     "min_child_weight": 80,
72     "reg_lambda": 50,
73     "gamma": 0.0,
74     "tree_method": "hist",
75     "max_bin": 256,
76     "eval_metric": "rmse",
77     "objective": "reg:squarederror",
78     "n_jobs": -1,
79     "random_state": 42,
80 }
81 FINAL_MODEL_PARAMETERS = {
82     "manual": MANUAL_PARAMETERS,
83     "xgboost_base": XGBOOST_BASE_PARAMETERS,
84     "xgboost_improved": XGBOOST_IMPROVED_PARAMETERS,
85 }
```

```

86
87
88 def validate_final_comparison_config(config):
89     errors = []
90     for name, expected in FINAL_REQUIRED_VALUES.items():
91         value = config.get(name)
92         if value != expected:
93             errors.append(f"{name}={value!r}; esperado {expected!r}
94                             }.")
95     for model_name, parameters in FINAL_MODEL_PARAMETERS.items():
96         for name, expected in parameters.items():
97             value = config[model_name].get(name)
98             if value != expected:
99                 errors.append(
100                     f"{model_name}.{name}={value!r}; esperado {
101                         expected!r}."
102                 )
103
104     if errors:
105         details = "\n".join(f"- {error}" for error in errors)
106         raise ValueError(
107             "La comparación final no se ejecutará hasta restaurar
108             los parámetros "
109             f"congelados:\n{details}"
110         )
111     print("\nConfiguración final confirmada: se usarán los pará
112           metros congelados.")
113
114
115 def create_final_output_directory():
116     FINAL_COMPARISON_DIRECTORY.mkdir(parents=True, exist_ok=True)
117     run_id = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
118     output_dir = FINAL_COMPARISON_DIRECTORY / run_id
119     output_dir.mkdir()
120     return run_id, output_dir
121
122
123 def append_final_execution_history(run_id, output_dir, records,
124                                   total_duration_seconds):
125     history = pd.DataFrame(records)
126     history.insert(0, "ejecucion", run_id)
127     history.insert(1, "directorio", str(output_dir))
128     history.insert(2, "total_duration_seconds",
129                   total_duration_seconds)
130     history_path = FINAL_COMPARISON_DIRECTORY / "
131                   registro_ejecuciones.csv"
132     history.to_csv(
133         history_path,
134         mode="a",
135         header=not history_path.exists(),
136         index=False,

```

```

131     )
132
133
134 def initial_graphs(df):
135     delays_hist(df)
136     mean_delay_by_hour(df)
137     mean_delay_by_month(df)
138     mean_delay_by_week(df)
139     correlation_matrix(df)
140
141
142 def run_manual_model(train_df, val_df, test_df, config,
143                     evaluate_test=False):
144     X_train, y_train, X_val, y_val, X_test, y_test =
145         prepare_model_data(
146             train_df,
147             val_df,
148             test_df,
149             sample_fraction=config["sample_fraction"],
150             drop_columns=["DepDelay", "UniqueCarrier", "Origin", "Dest
151                           "],
152             normalize=True,
153             random_state=config["random_state"],
154         )
155     model_X_test = X_test if evaluate_test else None
156     model_y_test = y_test if evaluate_test else None
157     model, predictions, history = run_regression_model(
158         X_train,
159         y_train,
160         X_val,
161         y_val,
162         model_X_test,
163         model_y_test,
164         **config["manual"],
165     )
166     metrics = evaluate_all_splits(
167         y_train,
168         predictions["train"],
169         y_val,
170         predictions["val"],
171         model_y_test,
172         predictions.get("test"),
173     )
174     return model, predictions, history, metrics
175
176
177 def run_xgboost_model(train_df, val_df, test_df, config,
178                      parameters,
179                      add_time_categories, evaluate_test=False):
180     X_train, y_train, X_val, y_val, X_test, y_test, preprocessor =
181         prepare_model_data(

```

```

177     train_df,
178     val_df,
179     test_df,
180     sample_fraction=config["sample_fraction"],
181     drop_columns=["DepDelay"],
182     encode_categorical=True,
183     random_state=config["random_state"],
184     return_preprocessor=True,
185     add_time_categories=add_time_categories,
186 )
187 model_X_test = X_test if evaluate_test else None
188 model_y_test = y_test if evaluate_test else None
189 model, predictions, history = run_xgboost(
190     X_train,
191     y_train,
192     X_val,
193     y_val,
194     model_X_test,
195     model_y_test,
196     **parameters,
197 )
198 metrics = evaluate_all_splits(
199     y_train,
200     predictions["train"],
201     y_val,
202     predictions["val"],
203     model_y_test,
204     predictions.get("test"),
205 )
206 return model, predictions, history, metrics, preprocessor
207
208
209 def generate_individual_graphs(model_name, predictions, history,
210                                metrics, split):
211     graph_suffix = "Test_Final" if split == "test" else "
212     Validation"
213     label = model_name.replace(" ", "") + f"_{graph_suffix}"
214     y_evaluation = predictions[f"y_{split}"]
215     predicted_vs_actual(
216         y_evaluation,
217         predictions[split],
218         metrics[f"{split}_r2"],
219         dataset_name=label,
220     )
221     residual_plot(y_evaluation, predictions[split], dataset_name=
222         label)
223     error_distribution(y_evaluation, predictions[split],
224         dataset_name=label)
225
226     if model_name == "Manual":
227         filename = (

```

```

224         "manual_learning_curve_test_final"
225         if split == "test" else "manual_learning_curve"
226     )
227     plot_training_history(
228         history["epoch"],
229         history["train_mse"],
230         history["val_mse"],
231         xlabel="Época",
232         ylabel="MSE",
233         title="Evolución del MSE - Modelo Manual",
234         filename=filename,
235     )
236     return
237
238     filename_suffix = "test_final" if split == "test" else "
validation"
239     xgb_cost_evolution(
240         history["train_rmse"],
241         history["val_rmse"],
242         title=f"Evolución del RMSE - {model_name}",
243         filename=model_name.lower().replace(" ", "_")
244         + f"_learning_curve_{filename_suffix}",
245     )
246
247
248 def final_metrics_record(model_name, metrics, duration_seconds,
sample_fraction):
249     test_metrics = {
250         name.removeprefix("test_"): value
251         for name, value in metrics.items()
252         if name.startswith("test_")
253     }
254     return {
255         "modelo": model_name,
256         "sample_fraction": sample_fraction,
257         "duration_seconds": duration_seconds,
258         **test_metrics,
259     }
260
261
262 def create_prediction_comparison(y_test, predictions_by_model):
263     comparison = pd.DataFrame({
264         "observacion": range(1, len(y_test) + 1),
265         "valor_real": y_test.to_numpy(),
266     })
267     for model_name, predictions in predictions_by_model.items():
268         prediction_key = model_name.lower().replace(" ", "_")
269         comparison[f"prediccion_{prediction_key}"] = predictions
270     return comparison.head(10)
271
272

```

```

273 def generate_comparison_graphs(model_results, model_predictions,
    split,
274                                 dataset_name):
275     title_suffix = "Test final" if split == "test" else "
        Validation"
276     for metric, title in (
277         ("mae", "Comparación de MAE"),
278         ("rmse", "Comparación de RMSE"),
279         ("r2", "Comparación de R2"),
280         ("within_15", "Predicciones dentro de 15 minutos"),
281     ):
282         compare_models_bar(
283             model_results,
284             f"{split}_{metric}",
285             f"{title} - {title_suffix}",
286         )
287     predictions_for_plot = {
288         model_name: predictions[split]
289         for model_name, predictions in model_predictions.items()
290     }
291     first_model = next(iter(model_predictions))
292     binned_prediction_plot(
293         predictions_for_plot,
294         model_predictions[first_model][f"y_{split}"],
295         n_bins=20,
296         dataset_name=dataset_name,
297     )
298
299
300 def run_final_comparison(train_df, val_df, test_df, config,
    program_started):
301     records = []
302     model_results = {}
303     model_predictions = {}
304     improved_model = None
305     improved_preprocessor = None
306
307     print("\n" + "=" * 70)
308     print("COMPARACIÓN FINAL SOBRE TEST")
309     print("=" * 70)
310
311     started = time.perf_counter()
312     _, predictions, history, metrics = run_manual_model(
313         train_df,
314         val_df,
315         test_df,
316         config,
317         evaluate_test=True,
318     )
319     model_results["Manual"] = metrics
320     model_predictions["Manual"] = predictions

```

```
321     records.append(  
322         final_metrics_record(  
323             "Manual",  
324             metrics,  
325             time.perf_counter() - started,  
326             config["sample_fraction"],  
327         )  
328     )  
329     if config["generate_individual_graphs"]:  
330         generate_individual_graphs("Manual", predictions, history,  
331                                   metrics, "test")  
332  
333     for model_name, parameters, add_time_categories in (  
334         ("XGBoost base", config["xgboost_base"], False),  
335         ("XGBoost mejorado", config["xgboost_improved"], True),  
336     ):  
337         print(  
338             f"\nEntrenando {model_name}: {parameters['n_estimators']}"  
339             f"árboles "  
340             f"(avance cada 100 árboles)."  
341         )  
342         started = time.perf_counter()  
343         model, predictions, history, metrics, preprocessor =  
344             run_xgboost_model(  
345                 train_df,  
346                 val_df,  
347                 test_df,  
348                 config,  
349                 parameters,  
350                 add_time_categories,  
351                 evaluate_test=True,  
352             )  
353         model_results[model_name] = metrics  
354         model_predictions[model_name] = predictions  
355         records.append(  
356             final_metrics_record(  
357                 model_name,  
358                 metrics,  
359                 time.perf_counter() - started,  
360                 config["sample_fraction"],  
361             )  
362         )  
363         if config["generate_individual_graphs"]:  
364             generate_individual_graphs(  
365                 model_name,  
366                 predictions,  
367                 history,  
368                 metrics,  
369                 "test",  
370             )  
371         if model_name == "XGBoost mejorado":
```

```

369         improved_model = model
370         improved_preprocessor = preprocessor
371
372     comparison = create_comparison_dataframe(model_results, split=
373         "test")
374     if config["generate_comparison_graphs"]:
375         print("\nGenerando gráficas comparativas de test...")
376         generate_comparison_graphs(
377             model_results,
378             model_predictions,
379             "test",
380             "Test_Final_XGBoost",
381         )
382     run_id, output_dir = create_final_output_directory()
383     comparison.to_csv(output_dir / "comparacion_test.csv", index=
384         False)
385     final_predictions = {
386         model_name: predictions["test"]
387         for model_name, predictions in model_predictions.items()
388     }
389     prediction_comparison = create_prediction_comparison(
390         test_df["DepDelay"],
391         final_predictions,
392     )
393     prediction_comparison.to_csv(
394         output_dir / "predicciones_test_primeras_10.csv",
395         index=False,
396         float_format="%.4f",
397     )
398     (output_dir / "configuraciones.json").write_text(
399         json.dumps(
400             {
401                 "manual": config["manual"],
402                 "xgboost_base": config["xgboost_base"],
403                 "xgboost_mejorado": config["xgboost_improved"],
404             },
405             indent=2,
406             ensure_ascii=False,
407         ),
408         encoding="utf-8",
409     )
410     improved_model.save_model(str(output_dir / "xgboost_mejorado.
411         json"))
412     with open(output_dir / "preprocessor_xgboost_mejorado.pkl", "
413         wb") as file:
414         pickle.dump(improved_preprocessor, file, protocol=pickle.
415             HIGHEST_PROTOCOL)
416     total_duration_seconds = time.perf_counter() - program_started
417     (output_dir / "metricas.json").write_text(
418         json.dumps(
419             {

```

```

415         "total_duration_seconds": total_duration_seconds,
416         "models": records,
417     },
418     indent=2,
419     ensure_ascii=False,
420 ),
421     encoding="utf-8",
422 )
423 append_final_execution_history(
424     run_id,
425     output_dir,
426     records,
427     total_duration_seconds,
428 )
429
430 print(comparison.to_string(index=False))
431 print("\nPrimeras 10 predicciones sobre test:")
432 print(prediction_comparison.to_string(index=False))
433 print(f"\nTiempo total del programa: {total_duration_seconds /
434       60:.2f} minutos")
435
436 print(f"\nResultados finales guardados en: {output_dir}")
437
438 def run_experiment():
439     program_started = time.perf_counter()
440     config = {
441         "sample_fraction": 1.0,
442         "shuffle_data": True,
443         "random_state": 42,
444         "run_final_comparison": True,
445         "run_manual": False,
446         "run_xgboost_base": False,
447         "run_xgboost_improved": False,
448         "generate_basic_graphs": True,
449         "generate_individual_graphs": True,
450         "generate_comparison_graphs": True,
451         "manual": MANUAL_PARAMETERS.copy(),
452         "xgboost_base": XGBOOST_BASE_PARAMETERS.copy(),
453         "xgboost_improved": XGBOOST_IMPROVED_PARAMETERS.copy(),
454     }
455
456     if config["run_final_comparison"]:
457         validate_final_comparison_config(config)
458
459     print("\n" + "=" * 70)
460     print("ETL")
461     print("=" * 70)
462     df, skip_transform = extract_data()
463     if not skip_transform:
464         df = transform_data(df)
465     _, train_df, val_df, test_df = split_data(

```

```

465     df,
466     shuffle=config["shuffle_data"],
467     random_state=config["random_state"],
468 )
469
470 print("\nDatos separados:")
471 print(f"Train:         {len(train_df):,}")
472 print(f"Validation: {len(val_df):,}")
473 print(f"Test reservado: {len(test_df):,}")
474
475 if config["generate_basic_graphs"]:
476     print("\nGenerando gráficos iniciales...")
477     initial_graphs(df)
478     print("Gráficos iniciales terminados.")
479
480 if config["run_final_comparison"]:
481     run_final_comparison(train_df, val_df, test_df, config,
482                          program_started)
483     return
484
485 print(
486     f"\nTamaño efectivo del entrenamiento ({config['
487     sample_fraction'] * 100}%): "
488     f"{int(len(train_df) * config['sample_fraction']):,}"
489 )
490 model_results = {}
491 model_predictions = {}
492 model_histories = {}
493 trained_models = {}
494 fitted_preprocessors = {}
495
496 if config["run_manual"]:
497     print("\n" + "=" * 70)
498     print("REGRESIÓN MANUAL")
499     print("=" * 70)
500     model, predictions, history, metrics = run_manual_model(
501         train_df,
502         val_df,
503         test_df,
504         config,
505     )
506     trained_models["Manual"] = model
507     model_results["Manual"] = metrics
508     model_predictions["Manual"] = predictions
509     model_histories["Manual"] = history
510     if config["generate_individual_graphs"]:
511         generate_individual_graphs(
512             "Manual",
513             predictions,
514             history,
515             metrics,

```

```

514         "val",
515     )
516
517     for model_name, config_key, run_key, add_time_categories in (
518         ("XGBoost base", "xgboost_base", "run_xgboost_base", False
519         ),
520         ("XGBoost mejorado", "xgboost_improved", "
521         run_xgboost_improved", True),
522     ):
523         if not config[run_key]:
524             continue
525         print("\n" + "=" * 70)
526         print(model_name.upper())
527         print("=" * 70)
528         model, predictions, history, metrics, preprocessor =
529         run_xgboost_model(
530             train_df,
531             val_df,
532             test_df,
533             config,
534             config[config_key],
535             add_time_categories,
536         )
537         trained_models[model_name] = model
538         model_results[model_name] = metrics
539         model_predictions[model_name] = predictions
540         model_histories[model_name] = history
541         fitted_preprocessors[model_name] = preprocessor
542         if config["generate_individual_graphs"]:
543             generate_individual_graphs(
544                 model_name,
545                 predictions,
546                 history,
547                 metrics,
548                 "val",
549             )
550
551     if not model_results:
552         print("\nNo se ejecutó ningún modelo.")
553         return
554
555     comparison = create_comparison_dataframe(model_results, split=
556         "val")
557     print("\n" + "=" * 90)
558     print("COMPARACIÓN DE MODELOS - VALIDATION")
559     print("=" * 90)
560     print(comparison.to_string(index=False))
561     diagnostic = create_diagnostic_dataframe(model_results)
562     print_diagnostic_summary(model_results)
563
564     if config["generate_comparison_graphs"]:

```

```
561     generate_comparison_graphs(  
562         model_results,  
563         model_predictions,  
564         "val",  
565         "Validation",  
566     )  
567  
568     save_experiment(  
569         {  
570             "config": config,  
571             "metrics": model_results,  
572             "histories": model_histories,  
573             "comparison": comparison,  
574             "diagnostic": diagnostic,  
575             "models": trained_models,  
576             "preprocessors": fitted_preprocessors,  
577         }  
578     )  
579     print("\nExperimento terminado.")
```